

# Design and Analysis of Algorithms

## Problem Set 1 — Complete Solutions

September 2024

### Contents

|          |                                      |           |
|----------|--------------------------------------|-----------|
| <b>1</b> | <b>Divide and Conquer</b>            | <b>2</b>  |
| <b>2</b> | <b>Augmented Binary Search Trees</b> | <b>10</b> |
| <b>3</b> | <b>Greedy Strategy</b>               | <b>20</b> |

# 1 Divide and Conquer

## Problem 1: Counting Inversions

Given an array  $A$  of  $n$  distinct numbers, a pair  $(i, j)$  with  $0 \leq i < j \leq n - 1$  is an *inversion* if  $A[i] > A[j]$ . Design an  $O(n \log n)$  algorithm to count all inversions.

**Algorithmic Intuition.** An inversion is precisely a pair of elements that are out of relative order. Merge sort naturally sorts by repeatedly merging two already-sorted halves; during the merge step, whenever an element from the right half is placed before some remaining elements of the left half, each such remaining left element forms an inversion with it. We therefore augment merge sort to count these cross-inversions for free while it sorts.

---

**Algorithm 1:** CountInversions( $A, p, r$ ) — returns (sorted array, inversion count)

---

```
1 if  $p \geq r$  then
2   return ( $A[p..r], 0$ )
3  $q \leftarrow \lfloor (p + r) / 2 \rfloor$ ;
4 ( $L, c_1$ )  $\leftarrow$  CountInversions( $A, p, q$ );
5 ( $R, c_2$ )  $\leftarrow$  CountInversions( $A, q + 1, r$ );
6 ( $M, c_3$ )  $\leftarrow$  MergeAndCount( $L, R$ );
7 return ( $M, c_1 + c_2 + c_3$ );
```

---

---

**Algorithm 2:** MergeAndCount( $L, R$ )

---

```
1  $i \leftarrow 0; j \leftarrow 0; \text{count} \leftarrow 0; M \leftarrow$  empty array of size  $|L| + |R|$ ;
2 for  $k \leftarrow 0$  to  $|L| + |R| - 1$  do
3   if  $j \geq |R|$  or ( $i < |L|$  and  $L[i] \leq R[j]$ ) then
4      $M[k] \leftarrow L[i]; i \leftarrow i + 1$ ;
5   else
6      $M[k] \leftarrow R[j]; j \leftarrow j + 1$ ;
7      $\text{count} \leftarrow \text{count} + (|L| - i);$  // every remaining element of  $L$  inverts with
       $R[j]$ 
8 return ( $M, \text{count}$ );
```

---

**Complexity Analysis.** Let  $T(n)$  be the running time on an array of size  $n$ . The merge step is  $O(n)$ , giving

$$T(n) = 2T(n/2) + O(n).$$

By the Master Theorem (Case 2,  $a = 2, b = 2, f(n) = \Theta(n) = \Theta(n^{\log_b a})$ ),  $T(n) = O(n \log n)$ .

**Proof of Correctness.** We prove by strong induction on the size of the subarray  $A[p..r]$  that CountInversions returns the correctly sorted subarray together with the exact number of inversions within  $A[p..r]$ .

*Base case.* If  $p \geq r$ , the subarray has at most one element, which is trivially sorted with 0 inversions.

*Inductive step.* Assume the claim holds for all subarrays of length  $< n$ . For a subarray of length  $n$ , the recursive calls on  $A[p..q]$  and  $A[q + 1..r]$  correctly return  $L, R$  sorted with counts  $c_1, c_2$  equal to the number of inversions entirely within the left half and entirely within the right half, respectively (by the inductive hypothesis). Every inversion in  $A[p..r]$  falls into exactly one of

three disjoint categories: (i) both indices in  $[p, q]$  — counted by  $c_1$ ; (ii) both indices in  $[q + 1, r]$  — counted by  $c_2$ ; (iii) one index in each half — a *split inversion*. We claim MergeAndCount computes exactly the number of split inversions,  $c_3$ .

Since  $L$  and  $R$  are each individually sorted (by the inductive hypothesis), an original left-half element  $x = L[i']$  and right-half element  $y = R[j']$  form a split inversion iff  $x > y$ , i.e., iff  $x$  appears after  $y$  in sorted order. During the merge, whenever we output  $R[j]$  (i.e.  $R[j] < L[i]$  for the current pointer  $i$ ), because  $L$  is sorted, we know  $R[j] < L[i] \leq L[i + 1] \leq \dots \leq L[|L| - 1]$ ; hence  $R[j]$  is smaller than *every* remaining element of  $L$ , each of which therefore forms a split inversion with  $R[j]$ . There are exactly  $|L| - i$  such elements at that moment, which is precisely the increment added. No split inversion is double-counted, since each  $R[j]$  is processed exactly once, and no split inversion is missed, since every  $L[i]$  that ultimately precedes some  $R[j]$  in original index but follows it in value is accounted for at the step where that particular  $R[j]$  is merged in. Hence  $c_3$  equals the true number of split inversions, and  $c_1 + c_2 + c_3$  is the total inversion count of  $A[p..r]$ . ■

## Problem 2: Missing Element in an Arithmetic Progression

Given an array representing an arithmetic progression (AP) with exactly one element missing, find the missing element in  $O(\log n)$  time.

**Algorithmic Intuition.** The input array has  $n$  entries and is obtained from a full AP of  $n + 1$  terms  $t_0, t_1, \dots, t_n$  by deleting one term; as is standard for this problem we assume the deleted term is not  $t_0$  or  $t_n$  (if an end term were deleted the remaining array would itself be a perfect AP and the "missing" term would not be determined by the data). Consequently  $A[0] = t_0$  and  $A[n - 1] = t_n$ , so  $A[n - 1] - A[0] = t_n - t_0 = n \cdot d$  and the common difference is recovered exactly as

$$d = \frac{A[n - 1] - A[0]}{n},$$

using integer/rational arithmetic. In a perfect AP starting at  $A[0]$ , we would have  $A[i] = A[0] + i \cdot d$  for all  $i$ . Once a single element is removed, every index at or after the gap is shifted, so  $A[i] - (A[0] + i \cdot d) = 0$  for indices strictly before the gap, and  $= -d$  (a constant nonzero offset) for indices at or after the gap. This is a monotone "step function" property that we can binary search on.

---

**Algorithm 3:** FindMissing( $A[0..n-1]$ )

---

**Input** :  $A$ : array of  $n$  elements, an AP of  $n+1$  terms with one removed

```
1  $d \leftarrow (A[n-1] - A[0])/n$ ;  
2  $lo \leftarrow 0$ ;  $hi \leftarrow n-1$ ;  
3 if  $d = 0$  then  
4   return  $A[0]$  ; // degenerate constant-difference edge case  
5 while  $lo \leq hi$  do  
6    $mid \leftarrow \lfloor (lo + hi)/2 \rfloor$ ;  
7   if  $A[mid] = A[0] + mid \cdot d$  then  
8      $lo \leftarrow mid + 1$  ; // gap is to the right  
9   else  
10    if  $mid = 0$  or  $A[mid-1] = A[0] + (mid-1) \cdot d$  then  
11      return  $A[0] + mid \cdot d$  ; // gap found between  $mid-1$  and  $mid$   
12    else  
13       $hi \leftarrow mid - 1$  ; // gap is to the left  
14 return  $A[0] + (n-1) \cdot d$  ; // unreachable under the stated assumption; defensive  
    fallback
```

---

**Complexity Analysis.** Each iteration performs  $O(1)$  work and halves the search range  $[lo, hi]$ , giving the recurrence  $T(n) = T(n/2) + O(1) = O(\log n)$  by the Master Theorem (Case 1).

**Proof of Correctness.** Define predicate  $P(i) \equiv [A[i] = A[0] + i \cdot d]$ . Because exactly one element is missing,  $P(i)$  is TRUE for all indices before the gap and FALSE for all indices at/after the gap —  $P$  is monotonically non-increasing over  $i \in [0, n-1]$  (true→false transition exactly once). This is precisely the binary-search invariant: at each step, if  $P(mid)$  holds, the gap must lie in  $(mid, hi]$ , so we discard the left part; otherwise the gap lies in  $[lo, mid]$ , and we check whether  $mid$  is the exact boundary (i.e.  $P(mid-1)$  holds) — if so we have located the missing value  $A[0] + mid \cdot d$ , otherwise we recurse left. The loop invariant "the gap boundary lies within  $[lo, hi]$ " is maintained by each branch and the search range strictly shrinks, so the algorithm terminates having correctly identified the unique missing value. ■

### Problem 3: Median of Two Sorted Arrays

Given two sorted arrays  $A, B$  of size  $n$  each, find the median of the merged array of size  $2n$  in  $O(\log n)$  time.

**Algorithmic Intuition.** We binary search for a *partition* of the combined array rather than merging it. A partition splits  $A$  into  $A[0..i-1] \mid A[i..n-1]$  and  $B$  into  $B[0..j-1] \mid B[j..n-1]$  such that the left parts together contain exactly  $n$  of the  $2n$  elements ( $i+j=n$ ) and every element on the left is  $\leq$  every element on the right. Once such a balanced partition is found, the median is computable in  $O(1)$  from the four boundary elements.

---

**Algorithm 4:** FindMedian( $A[0..n-1], B[0..n-1]$ )

---

```
1  $lo \leftarrow 0; hi \leftarrow n;$ 
2 while  $lo \leq hi$  do
3    $i \leftarrow \lfloor (lo + hi)/2 \rfloor;$  // partition point in A
4    $j \leftarrow n - i;$  // partition point in B, forced so left half has size n
5    $A_L \leftarrow (i = 0) ? -\infty : A[i-1];$   $A_R \leftarrow (i = n) ? +\infty : A[i];$ 
6    $B_L \leftarrow (j = 0) ? -\infty : B[j-1];$   $B_R \leftarrow (j = n) ? +\infty : B[j];$ 
7   if  $A_L \leq B_R$  and  $B_L \leq A_R$  then
8     return  $(\max(A_L, B_L) + \min(A_R, B_R))/2;$  // valid partition found; 2n is
      even
9   else
10    if  $A_L > B_R$  then
11       $hi \leftarrow i - 1$ 
12    else
13       $lo \leftarrow i + 1$ 
```

---

**Complexity Analysis.** The search halves  $[lo, hi]$  (of initial size  $n$ ) each iteration and does  $O(1)$  work per iteration, so  $T(n) = T(n/2) + O(1) = O(\log n)$ .

**Proof of Correctness.** *Invariant.* At every iteration  $i + j = n$  exactly, so the multiset  $\{A[0..i-1]\} \cup \{B[0..j-1]\}$  always has exactly  $n$  elements — this is precisely the left half of the merged  $2n$ -element array *once the partition is valid*.

A partition is valid iff  $A_L \leq B_R$  and  $B_L \leq A_R$ : since  $A, B$  are individually sorted,  $A_L \leq A_R$  and  $B_L \leq B_R$  always hold; the two cross-conditions additionally guarantee every left element is  $\leq$  every right element, i.e. the left  $n$  elements collected are exactly the  $n$  smallest of the union. Hence  $\max(A_L, B_L)$  is the largest of the lower half and  $\min(A_R, B_R)$  the smallest of the upper half; since  $2n$  is even, the median is their average.

*Monotonicity for binary search.* As  $i$  increases,  $A_L$  is non-decreasing and  $A_R$  is non-decreasing; correspondingly  $j = n - i$  decreases, so  $B_L$  is non-increasing and  $B_R$  is non-increasing. If  $A_L > B_R$ , the segment  $A[0..i-1]$  contains a value exceeding some value in  $B[j..n-1]$  that is supposed to be in the right half —  $i$  is too large, so we must decrease  $i$  (search left,  $hi \leftarrow i - 1$ ). Symmetrically, if  $B_L > A_R$ ,  $i$  is too small and we increase it ( $lo \leftarrow i + 1$ ). This shows the search space always contains a valid  $i^*$  and the two branches move strictly toward it, so termination yields the correct partition. ■

#### Problem 4: Polynomial Coefficients from Zeros

Given values  $z_0, \dots, z_{n-1}$  (possibly repeated), find the coefficients of the degree- $< n$  polynomial  $P(x)$  with zeros exactly at these points (with multiplicity), in  $O(n \log^2 n)$  time.

**Algorithmic Intuition.**  $P(x)$  has a root at  $z_j$  (with the appropriate multiplicity) iff  $(x - z_j)$  divides  $P(x)$ ; since the  $z_j$  are given with repetition already reflecting multiplicity, we simply need

$$P(x) = \prod_{j=0}^{n-1} (x - z_j).$$

Multiplying all  $n$  linear factors pairwise in a balanced (divide-and-conquer) fashion — rather than sequentially, which costs  $O(n^2)$  — lets us exploit fast polynomial multiplication via the FFT at each level.

---

**Algorithm 5:** BuildPoly( $z[lo..hi]$ ) — returns coefficient vector of  $\prod_{k=lo}^{hi} (x - z_k)$

---

```

1 if  $lo = hi$  then
2   return  $[-z_{lo}, 1]$  ; // the polynomial  $x - z_{lo}$ 
3  $mid \leftarrow \lfloor (lo + hi)/2 \rfloor$ ;
4  $P_L \leftarrow \text{BuildPoly}(z[lo..mid])$ ;
5  $P_R \leftarrow \text{BuildPoly}(z[mid + 1..hi])$ ;
6 return FFTMultiply( $P_L, P_R$ ) ; //  $O(k \log k)$  for degree- $k$  product via FFT

```

---

**Complexity Analysis.** At recursion depth  $\ell$  (root at depth 0), there are  $2^\ell$  subproblems, each multiplying two polynomials of degree  $\Theta(n/2^\ell)$ , costing  $O((n/2^\ell) \log(n/2^\ell))$  via FFT (since FFT multiplication of two degree- $k$  polynomials is  $O(k \log k)$ ). The total work at depth  $\ell$  is

$$2^\ell \cdot O\left(\frac{n}{2^\ell} \log \frac{n}{2^\ell}\right) = O(n \log n).$$

There are  $O(\log n)$  levels, giving total time  $T(n) = O(n \log n) \cdot O(\log n) = O(n \log^2 n)$ , matching the recurrence  $T(n) = 2T(n/2) + O(n \log n)$  solved via the recursion-tree method.

**Proof of Correctness.** By induction on subarray length: the base case  $z[lo..lo]$  trivially returns the correct degree-1 polynomial  $x - z_{lo}$ . For the inductive step, by hypothesis  $P_L = \prod_{k=lo}^{mid} (x - z_k)$  and  $P_R = \prod_{k=mid+1}^{hi} (x - z_k)$  exactly. Polynomial multiplication is associative and commutative over the ring of polynomials, and FFTMultiply computes the exact coefficient-vector product of  $P_L$  and  $P_R$  (standard correctness of convolution via the Fast Fourier Transform and its inverse, since pointwise multiplication of the polynomials' value representations at the  $2^k$ -th roots of unity, followed by interpolation, recovers exactly the coefficients of the product polynomial of degree  $\deg P_L + \deg P_R$ ). Hence  $\text{BuildPoly}(z[lo..hi]) = P_L \cdot P_R = \prod_{k=lo}^{hi} (x - z_k)$ , completing the induction; the top-level call returns  $\prod_{k=0}^{n-1} (x - z_k) = P(x)$ , which by construction has exactly the roots  $z_0, \dots, z_{n-1}$  counted with multiplicity, and no others, since a nonzero polynomial over a field has no roots beyond those forced by its factorization. ■

## Problem 5: Toeplitz Matrices

A Toeplitz matrix  $A = (a_{i,j})$  satisfies  $a_{i,j} = a_{i-1,j-1}$  for all valid  $i, j \geq 2$ ; i.e. every diagonal is constant.

**(a) Sums and Products. Sum:** Yes. If  $A, B$  are Toeplitz,  $(A + B)_{i,j} = a_{i,j} + b_{i,j}$  depends only on  $i - j$  for each matrix separately, hence so does their sum:  $(A + B)_{i,j} = f_A(i - j) + f_B(i - j) = f_{A+B}(i - j)$ . So  $A + B$  is Toeplitz. **Product:** No, not in general. A single explicit counterexample suffices:

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix}, \quad AB = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}.$$

Both  $A$  and  $B$  are Toeplitz (each diagonal of each is constant). Their product is not: the sub-diagonal entries of  $AB$  are  $(AB)_{2,1} = 0$  and  $(AB)_{3,2} = 1$ , which lie on the same diagonal ( $i - j = 1$ ) yet differ. Hence the product of two Toeplitz matrices need not be Toeplitz.

**(b) Compact Representation and  $O(n)$  Addition.** An  $n \times n$  Toeplitz matrix is fully determined by its first row and first column, i.e. by the  $2n - 1$  values on the diagonals, indexed by  $d = i - j \in \{-(n - 1), \dots, n - 1\}$ : store the array  $t[-(n - 1) \dots n - 1]$  with  $t[d] = a_{i,j}$  for any  $i - j = d$ . Addition of two Toeplitz matrices, represented this way, is simply  $t_{A+B}[d] = t_A[d] + t_B[d]$  for all  $2n - 1$  values of  $d$  — an  $O(n)$  operation (compare to  $O(n^2)$  for dense matrices).

**(c)  $O(n \log n)$  Toeplitz  $\times$  Vector.**

**Intuition.** A Toeplitz matrix-vector product is a (sliding-window) *linear convolution* of the diagonal vector  $t$  with the input vector  $x$ :  $y_i = \sum_j t[i - j]x_j$ . Linear convolution embeds into *circular* convolution of a larger, power-of-two-padded vector, which the FFT computes in  $O(n \log n)$ .

---

**Algorithm 6:** ToeplitzVec( $t[-(n - 1) \dots n - 1], x[0 \dots n - 1]$ )

---

```

1 Let  $m$  be the smallest power of 2 with  $m \geq 2n$ ;
2 Build  $c[0 \dots m - 1]$ : a circulant generator with  $c[k] = t[k]$  for  $k = 0, \dots, n - 1$ ,
    $c[m - k] = t[-k]$  for  $k = 1, \dots, n - 1$ , zero elsewhere;
3 Pad  $x$  to length  $m$  with zeros:  $\hat{x}[0 \dots n - 1] = x$ ,  $\hat{x}[n \dots m - 1] = 0$ ;
4  $\hat{C} \leftarrow \text{FFT}(c)$ ;  $\hat{X} \leftarrow \text{FFT}(\hat{x})$ ; //  $O(m \log m)$  each
5  $\hat{Y}[k] \leftarrow \hat{C}[k] \cdot \hat{X}[k]$  for all  $k$ ; // pointwise,  $O(m)$ 
6  $y \leftarrow \text{InverseFFT}(\hat{Y})$ ;
7 return  $y[0 \dots n - 1]$ ;
```

---

This runs in  $O(n \log n)$  since  $m = \Theta(n)$ .

**Correctness.** Embedding the (non-circular) Toeplitz product into a circulant matrix of size  $m \geq 2n - 1$  guarantees that the circular wrap-around terms land in the padded (unused) zero region and do not corrupt entries  $y[0 \dots n - 1]$ ; extracting the FFT-based circular convolution's first  $n$  entries therefore yields exactly  $y_i = \sum_{j=0}^{n-1} t[i - j]x_j = (Ax)_i$ , the true Toeplitz-vector product, by the standard convolution theorem (circular convolution via FFT computes the DFT-domain pointwise product, whose inverse DFT is the exact convolution of the padded sequences).

**(d) Multiplying Two  $n \times n$  Toeplitz Matrices.** Represent  $A, B$  compactly as in (b). Their product  $C = AB$  is, in general, *not* Toeplitz (part (a)), so  $C$  must be stored densely —  $\Theta(n^2)$  entries, an unavoidable lower bound on output size. We compute  $C$  column-by-column: the  $k$ -th column of  $C$  equals  $A \cdot (k\text{-th column of } B)$ , a Toeplitz-matrix-times-vector product solvable in  $O(n \log n)$  by

---

**Algorithm 7:** ToeplitzMatMul( $A, B$ )

---

```

part (c). 1 for  $k \leftarrow 0$  to  $n - 1$  do
2    $C[:, k] \leftarrow \text{ToeplitzVec}(t_A, B[:, k])$ ; //  $O(n \log n)$  per column
3 return  $C$ ;
```

---

**Running time:**  $n$  columns  $\times$   $O(n \log n)$  per column  $= O(n^2 \log n)$ , a substantial improvement over the naive  $O(n^3)$  dense product.

## Problem 6: Local Minimum in a Complete Binary Tree

Given an  $n$ -node ( $n = 2^d - 1$ ) complete binary tree with distinct real labels accessible only by probing, find a local minimum (a node no larger than any neighbor) using  $O(\log n)$  probes.

**Algorithmic Intuition.** This is a discrete analogue of gradient descent. Start at the root; at each visited node, probe its (up to two) children. If the current node is smaller than both children, it is a local minimum (a leaf compared against only its parent, already known, also terminates correctly) — done. Otherwise, move to whichever neighbor has a strictly smaller value than the current node, and repeat. Because we always move to a strictly smaller value along a root-to-leaf path, and the tree has depth  $O(\log n)$ , the walk must terminate within  $O(\log n)$  steps.

---

**Algorithm 8:** FindLocalMin( $T$ )

---

```

1  $v \leftarrow \text{root}(T)$ ;
2 while true do
3    $\ell \leftarrow \text{leftChild}(v)$ ;  $r \leftarrow \text{rightChild}(v)$  ;           // probe;  $\pm\infty$  if absent
4   if  $x_v \leq x_\ell$  and  $x_v \leq x_r$  then
5     return  $v$  ; //  $v$ 's parent value was already known smaller at the step we
        moved here, or  $v$  is the root
6   else
7      $v \leftarrow \arg \min(\{x_\ell, x_r\} \setminus \{\pm\infty\})$  ;           // move to the smaller present child

```

---

**Complexity Analysis.** Each loop iteration does  $O(1)$  probes (two child probes). The path traced strictly decreases in value at each step (we only move to a child smaller than the current node), so it can never revisit a node, and since it always moves *downward* (root to a descendant, never to the parent), the number of steps is bounded by the tree's height,  $d - 1 = O(\log n)$ . Total probes:  $O(\log n)$ .

**Proof of Correctness.** *Invariant.* At the moment we arrive at node  $v$  (other than the root), we know  $x_v < x_{\text{parent}(v)}$  (that is why we moved there), so  $v$  is automatically  $\leq$  its parent's value; we still must probe  $v$ 's children to check the local-minimum condition fully, which the algorithm does.

*Termination implies correctness.* The algorithm only returns a node  $v$  once it has confirmed by direct probe that  $x_v \leq x_\ell$  and  $x_v \leq x_r$  for its (present) children, and by the invariant above  $x_v \leq x_{\text{parent}(v)}$  whenever  $v \neq \text{root}$  (and the root has no parent constraint to satisfy). Hence the returned node is  $\leq$  all of its tree-neighbors, i.e. a genuine local minimum.

*Termination is guaranteed.* The sequence of values visited,  $x_{v_0} > x_{v_1} > x_{v_2} > \dots$ , is strictly decreasing (by construction we only step to a strictly smaller child), so no node repeats; since the tree is finite, the walk must terminate, and since it only moves to children, it terminates within  $O(\log n)$  steps bounded by the height of the complete binary tree. ■

## Problem 7: Maximum Stock Profit (Divide and Conquer)

Given prices  $A[1..n]$ , find  $\max_{i \leq j} (A[j] - A[i])$  using divide and conquer.

**Algorithmic Intuition.** Split the array at the midpoint  $m$ . The optimal buy/sell pair either lies entirely in the left half, entirely in the right half, or straddles the midpoint (buy in the left half, sell in the right half). The first two cases are handled recursively; the straddling case is computed by finding the minimum price in the left half and the maximum price in the right half.



---

**Algorithm 9:** MaxProfit( $A, i, j$ ) — returns (profit, min, max) over  $A[i..j]$

---

```

1 if  $i = j$  then
2   return  $(0, A[i], A[i])$ ;    // single day: no trade possible, extremes trivial
3  $m \leftarrow \lfloor (i + j)/2 \rfloor$ ;
4  $(P_L, \min_L, \max_L) \leftarrow \text{MaxProfit}(A, i, m)$ ;
5  $(P_R, \min_R, \max_R) \leftarrow \text{MaxProfit}(A, m + 1, j)$ ;
6  $P_{\text{cross}} \leftarrow \max(0, \max_R - \min_L)$ ;           // buy in left, sell in right
7  $P \leftarrow \max(P_L, P_R, P_{\text{cross}})$ ;
8 return  $(P, \min(\min_L, \min_R), \max(\max_L, \max_R))$ ;

```

---

**Complexity Analysis.** Each call does  $O(1)$  work besides the two recursive calls (combining is constant-time given the returned min/max), so

$$T(n) = 2T(n/2) + O(1).$$

Since each call returns the minimum and the maximum of its subarray along with the best profit, the combine step is  $O(1)$  and no rescanning is needed. By the Master Theorem (Case 1, since  $f(n) = O(1) = O(n^{\log_2 2^{-\epsilon}})$ ),  $T(n) = O(n)$ , which is optimal, as every element must be inspected. (If the min and max are instead recomputed by a linear scan at each level, the recurrence becomes  $T(n) = 2T(n/2) + O(n)$  and the bound is  $O(n \log n)$ .)

**Proof of Correctness.** By strong induction on interval length. *Base case:* a single day has no possible trade, profit 0;  $\min = \max = A[i]$ , trivially correct. *Inductive step:* assume correctness for all intervals shorter than  $[i, j]$ . Every valid trade  $(i^*, j^*)$  with  $i \leq i^* \leq j^* \leq j$  falls into exactly one of three cases: (1)  $j^* \leq m$  — entirely left, captured by  $P_L$  via the inductive hypothesis; (2)  $i^* \geq m + 1$  — entirely right, captured by  $P_R$ ; (3)  $i^* \leq m < j^*$  — straddling. For case (3), profit  $= A[j^*] - A[i^*] \leq \max_R - \min_L$  since  $A[j^*] \leq \max_R$  (as  $j^*$  is in the right half) and  $A[i^*] \geq \min_L$  (as  $i^*$  is in the left half); moreover this upper bound  $\max_R - \min_L$  is itself *achieved* by the specific pair (index of  $\min_L$ , index of  $\max_R$ ), which is itself a valid straddling trade (with  $\geq 0$  profit taken via the  $\max(0, \cdot)$  guard, matching "profit may be 0 if we skip trading" implicitly since we can always choose  $i^* = j^*$  within a half for 0 profit — validated by the base case). Hence  $P_{\text{cross}}$  exactly equals the best straddling profit, and  $P = \max(P_L, P_R, P_{\text{cross}})$  is exactly the best profit over  $[i, j]$ , completing the induction. ■

## 2 Augmented Binary Search Trees

### Problem 1: Which Fields Support FindRank in $O(\log n)$ ?

Each node  $v$  of a height-balanced BST  $T$  on  $n$  elements normally stores  $\text{size}(v)$ , the number of nodes of the subtree  $T(v)$ . We must decide which of the following fields, stored at every node *instead of*  $\text{size}(v)$ , still allows  $\text{FINDRANK}(T, x)$  — the number of elements of  $T$  smaller than  $x$  — to be answered in  $O(\log n)$  while being maintainable under Insert, Delete and Rotate:

$$\text{size}(v), \quad \text{left-size}(v) = |T(\text{left}(v))|, \quad \text{subtree-rank}(v), \quad \text{rank}(v),$$

where  $\text{subtree-rank}(v)$  is the rank of  $v$  inside its own subtree  $T(v)$  and  $\text{rank}(v)$  is the rank of  $v$  in the whole tree  $T$ .

**The augmentation criterion (Lecture 4).** A field  $f$  is an *admissible augmentation* if

- (A)  $f(v)$  can be computed in  $O(1)$  from  $v$  itself together with the *stored fields*  $f(\text{left}(v))$  and  $f(\text{right}(v))$ .

Condition (A) is exactly what yields the update bound: an insertion or a deletion changes the subtree contents of only the  $O(\log n)$  nodes on one root-to-leaf path, and a rotation changes the subtrees of only the  $O(1)$  nodes it touches, so recomputing  $f$  bottom-up at those nodes by (A) restores every field in  $O(\log n)$  time per operation. A field violating (A) is not covered by this scheme: the value at a node is then not determined by the values its children store, and the update has no local rule to apply.

1.  $\text{size}(v)$ : **admissible, and it answers FindRank in  $O(\log n)$ .** Condition (A) holds because

$$\text{size}(v) = 1 + \text{size}(\text{left}(v)) + \text{size}(\text{right}(v)), \quad \text{size}(\text{NULL}) = 0,$$

a constant-time expression in the children's stored fields. Hence  $\text{size}$  is maintained in  $O(\log n)$  per Insert/Delete and in  $O(1)$  per rotation.

To answer  $\text{FINDRANK}(T, x)$  we run the ordinary BST search for  $x$  and accumulate a counter: following the *right* link of  $v$  passes over all of  $T(\text{left}(v))$  and  $v$  itself, all of which are smaller than  $x$ , so we add  $\text{size}(\text{left}(v)) + 1$ ; following the *left* link passes only over elements larger than  $x$ , so nothing is added.

---

**Algorithm 10:** FindRank( $T, x$ ) — number of elements of  $T$  smaller than  $x$

---

```

1  $v \leftarrow \text{root}(T)$ ;  $\text{rank} \leftarrow 0$ ;
2 while  $v \neq \text{NULL}$  and  $\text{key}(v) \neq x$  do
3   if  $x > \text{key}(v)$  then
4      $\text{rank} \leftarrow \text{rank} + \text{size}(\text{left}(v)) + 1$ ;
5      $v \leftarrow \text{right}(v)$ ;           // right link:  $T(\text{left}(v))$  and  $v$  are smaller than  $x$ 
6   else
7      $v \leftarrow \text{left}(v)$ ;           // left link: rank unchanged
8 if  $v \neq \text{NULL}$  then  $\text{rank} \leftarrow \text{rank} + \text{size}(\text{left}(v))$ ;           //  $x$  is present, at  $v$ 
9 ;
10 return  $\text{rank}$ ;
```

---

Equivalently, for a node  $x$  present in  $T$ ,

$$|\{u \in T : \text{key}(u) < \text{key}(x)\}| = \text{size}(\text{left}(x)) + \sum_{\substack{u \text{ ancestor of } x \\ x \in T(\text{right}(u))}} (\text{size}(\text{left}(u)) + 1),$$

which is precisely what the loop accumulates; the 1-indexed rank of  $x$  is this quantity +1.

*Correctness.* Let  $v_0, \dots, v_k$  be the nodes visited. We claim the invariant: on arriving at  $v_i$ , rank equals the number of elements of  $T \setminus T(v_i)$  that are smaller than  $x$ . It holds at the root ( $T \setminus T(v_0) = \emptyset$ ). At  $v_i$ , by the BST property every element of  $T(v_i)$  outside  $T(\text{right}(v_i)) \cup \{v_i\}$  is  $< \text{key}(v_i)$  and every element of  $T(\text{right}(v_i))$  is  $> \text{key}(v_i)$ . If  $x > \text{key}(v_i)$  the search proceeds into  $T(\text{right}(v_i))$ , and the elements of  $T(v_i)$  removed from consideration are exactly  $T(\text{left}(v_i)) \cup \{v_i\}$ , all smaller than  $x$ ; adding  $\text{size}(\text{left}(v_i)) + 1$  restores the invariant. If  $x < \text{key}(v_i)$  the removed elements are  $T(\text{right}(v_i)) \cup \{v_i\}$ , all larger than  $x$ , so nothing is added. When the loop stops with  $v = \text{NULL}$ ,  $x \notin T$  and the invariant gives the answer; when it stops at  $v$  with  $\text{key}(v) = x$ , the remaining elements of  $T(v)$  smaller than  $x$  are exactly those of  $T(\text{left}(v))$ , which the final line adds. ■

*Example.* Let  $T$  hold  $\{2, 4, 6, 8, 10, 12, 14\}$  with root 8, left child 4 (children 2, 6) and right child 12 (children 10, 14).  $\text{FINDRANK}(T, 10)$ : at 8, since  $10 > 8$  we add  $\text{size}(\text{left}(8)) + 1 = 3 + 1 = 4$  and go right; at 12, since  $10 < 12$  we go left with rank unchanged; at 10 the key matches and we add  $\text{size}(\text{left}(10)) = 0$ . The answer 4 is correct: 2, 4, 6, 8 are the elements smaller than 10.

*Complexity.* The search path of a height-balanced BST has length  $O(\log n)$  and each level costs  $O(1)$ , so  $\text{FINDRANK}$  runs in  $O(\log n)$  time using  $O(1)$  extra space; the augmentation costs one machine word per node, i.e.  $O(n)$  space, and  $O(\log n)$  extra time per Insert/Delete.

## 2. left-size( $v$ ): *not* admissible.

**Claim 2.1.** *There is no function  $g$  with  $\text{left-size}(v) = g(v, \text{left-size}(\text{left}(v)), \text{left-size}(\text{right}(v)))$  for all BSTs; i.e. left-size violates (A).*

*Proof.* Consider two trees. In  $T_1$ , the root  $x$  has a single child  $y$  on the left, and  $y$  is a leaf. In  $T_2$ , the root  $x$  again has  $y$  as its only child on the left, but  $y$  has a right child  $z$ , a leaf. In both trees  $\text{left-size}(\text{left}(x)) = \text{left-size}(y) = 0$  and  $\text{right}(x) = \text{NULL}$ , so the arguments of  $g$  are identical; yet  $\text{left-size}(x) = 1$  in  $T_1$  and  $\text{left-size}(x) = 2$  in  $T_2$ . Hence no such  $g$  exists. □

The same obstruction is visible at a rotation, which is where the field actually has to be repaired. Let  $x$  have left child  $y$ , let  $y$  have subtrees  $A$  (left) and  $B$  (right), and let  $C$  be the right subtree of  $x$ . Before the right rotation,  $\text{left-size}(x) = |A| + |B| + 1$ ; after it,  $x$ 's left subtree is  $B$ , so  $\text{left-size}(x)$  must become  $|B|$ . But  $|B|$  is a full subtree size, and the field stored at  $B$ 's root is only  $\text{left-size}(\text{root}(B))$ , which says nothing about  $|B|$  (take  $B$  to be a right path: every node of  $B$  stores 0, whatever  $|B|$  is). So the new value is not computable from the children's stored fields, and recovering  $|B|$  from the left-size fields alone requires walking the right spine of  $B$ , which is not  $O(1)$ .

*Remark (scope of the claim).* The claim above is exactly the failure of criterion (A): left-size is not determined by the fields its children store. It should not be over-read. Note also that  $\text{left-size}(v) = \text{size}(\text{left}(v))$  is precisely the quantity the  $\text{FINDRANK}$  descent reads, so *query* time is not the issue — maintenance is; and if one steps outside criterion (A) and allows a rotation-specific rule that inspects both nodes taking part in the rotation, the identity  $|B| = \text{left-size}(x) - \text{left-size}(y) - 1$  (evaluated before the rotation) does restore the field in  $O(1)$ . Under the criterion stated in the question, however, left-size is not an admissible augmentation, and  $\text{size}(v)$  — which stores the full subtree size and therefore satisfies (A) — is the field to use.

**3. subtree-rank( $v$ ): *not* admissible either.** By the BST property, the elements of  $T(v)$  smaller than  $\text{key}(v)$  are exactly those of  $T(\text{left}(v))$ , so

$$\text{subtree-rank}(v) = \text{left-size}(v) + 1.$$

The two fields therefore carry exactly the same information, and the counterexample of the previous claim transfers verbatim: in the trees  $T_1, T_2$  above, the children store identical subtree-rank values (1 at  $y$ , resp. 1 at  $y$  and 1 at  $z$ ) while subtree-rank( $x$ ) is 2 in  $T_1$  and 3 in  $T_2$ . Hence subtree-rank violates (A) for the same reason, and the same rotation analysis applies with every value shifted by one.

**4. rank( $v$ ) (global rank in  $T$ ): *not* admissible.** Here the field is not even a function of  $T(v)$ :  $\text{rank}(v) = 1 + |\{u \in T : \text{key}(u) < \text{key}(v)\}|$  depends on the entire tree, so (A) fails immediately. More decisively, the field cannot be kept correct within the required time bound at all.

**Claim 2.2.** *Maintaining an explicit rank( $v$ ) field at every node requires  $\Omega(n)$  field updates for some single insertions.*

*Proof.* Insert a new element smaller than all keys of  $T$ . Every one of the  $n$  existing nodes then has exactly one more element below it, so its global rank increases by exactly 1, and all  $n$  stored fields become stale. Any algorithm keeping all fields correct must write to all  $n$  of them, which is  $\Omega(n)$ , not  $O(\log n)$ . (Rotations, by contrast, do not change any global rank, since they do not change the set of keys; it is Insert/Delete that breaks the bound.)  $\square$

**Conclusion.** Only  $\text{size}(v)$  qualifies: it satisfies (A), is therefore maintained in  $O(\log n)$  per update and  $O(1)$  per rotation, and supports FINDRANK in  $O(\log n)$  by the descent above. The fields  $\text{left-size}(v)$  and  $\text{subtree-rank}(v)$  are not determined by the fields stored at the children and so are not admissible augmentations in the sense of (A), while  $\text{rank}(v)$  is worse still, needing  $\Theta(n)$  updates for a single insertion. Note carefully what is *not* being claimed: rank *queries* are perfectly answerable in  $O(\log n)$  — that is what part 1 does — what fails is storing the global rank at every node as the augmentation.

## Problem 2: Dynamic Bit Sequence with Range-XOR

Maintain a sequence of  $n$  bits under  $\text{Insert}(i, b)$ ,  $\text{Delete}(i)$ ,  $\text{Report-XOR}(j, k)$ , each in  $O(\log n)$ .

**Data Structure.** An *implicit* (position-keyed, not value-keyed) balanced BST — a balanced binary search tree ordered purely by in-order position, e.g. a treap with random priorities to guarantee  $O(\log n)$  expected height, or a Red-Black tree with an implicit key — where each node stores one bit.

**Augmented Fields.** For each node  $v$ :

- $\text{bit}(v)$ : the bit value stored at  $v$ .
- $\text{size}(v)$ : number of nodes in  $T(v)$  (for positional indexing).
- $\text{xor}(v)$ : XOR of all bits in  $T(v)$ , satisfying the invariant

$$\text{xor}(v) = \text{xor}(\text{left}(v)) \oplus \text{bit}(v) \oplus \text{xor}(\text{right}(v)).$$

|                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Operations.</b>                                                                                                                                                                                                                                                                                                                  | <b>Algorithm 11:</b> $\text{Split}(T, i)$ — splits $T$ (by position) into $(T_1, T_2)$ with $ T_1  = i$                                                                                                                |
|                                                                                                                                                                                                                                                                                                                                     | 1 Standard treap positional split, descending using size fields to locate the boundary in $O(\log n)$ , recomputing size, xor bottom-up for the $O(\log n)$ nodes on the split path;<br><b>2 return</b> $(T_1, T_2)$ ; |
| <hr/>                                                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                        |
| <b>Algorithm 12:</b> $\text{Join}(T_1, T_2)$ — merges two trees, all of $T_1$ before all of $T_2$                                                                                                                                                                                                                                   |                                                                                                                                                                                                                        |
| 1 Standard treap merge-by-priority, $O(\log n)$ , recomputing size, xor along the $O(\log n)$ -length merge path;<br><b>2 return</b> merged tree;                                                                                                                                                                                   |                                                                                                                                                                                                                        |
| <hr/>                                                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                        |
| <b>Algorithm 13:</b> $\text{Insert}(i, b)$                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                        |
| 1 $(T_1, T_2) \leftarrow \text{Split}(T, i)$ ;<br>2 $v \leftarrow$ new node with $\text{bit}(v) = b$ , $\text{size}(v) = 1$ , $\text{xor}(v) = b$ ;<br>3 $T \leftarrow \text{Join}(\text{Join}(T_1, \{v\}), T_2)$ ;                                                                                                                 |                                                                                                                                                                                                                        |
| <hr/>                                                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                        |
| <b>Algorithm 14:</b> $\text{Delete}(i)$                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                        |
| 1 $(T_1, T_{23}) \leftarrow \text{Split}(T, i - 1)$ ;<br>2 $(\{v\}, T_3) \leftarrow \text{Split}(T_{23}, 1)$ ;<br>3 $T \leftarrow \text{Join}(T_1, T_3)$ ;                                                                                                                                                                          |                                                                                                                                                                                                                        |
| <hr/>                                                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                        |
| <b>Algorithm 15:</b> $\text{Report-XOR}(j, k)$                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                        |
| 1 $(T_1, T_{23}) \leftarrow \text{Split}(T, j - 1)$ ;<br>2 $(T_2, T_3) \leftarrow \text{Split}(T_{23}, k - j + 1)$ ;<br>3 $\text{ans} \leftarrow \text{xor}(\text{root}(T_2))$ ;<br>4 $T \leftarrow \text{Join}(\text{Join}(T_1, T_2), T_3)$ ; <i>// restore the tree, no structural change to sequence</i><br><b>5 return</b> ans; |                                                                                                                                                                                                                        |

**Complexity Analysis.** A treap has expected height  $O(\log n)$ ; Split and Join each run in  $O(\log n)$  (expected) time, touching only the nodes on a single root-to-leaf path, at each of which size and xor are recomputed in  $O(1)$  from (already correct) children fields. Insert, Delete, and Report-XOR each invoke  $O(1)$  Split/Join calls, hence run in  $O(\log n)$  (expected/amortized; a deterministic  $O(\log n)$  worst-case bound follows identically using a weight-balanced or Red-Black positional tree instead of a treap).

**Proof of Correctness.** By structural induction on tree construction: the invariant  $\text{xor}(v) = \text{xor}(\text{left}(v)) \oplus \text{bit}(v) \oplus \text{xor}(\text{right}(v))$  holds at leaves trivially ( $\text{xor}(v) = \text{bit}(v)$ , empty children contribute the XOR identity 0) and is preserved by every Split/Join/Insert/Delete since each recomputes exactly this formula bottom-up along the path of modified nodes; therefore it holds globally after every operation. Since XOR is associative and commutative, and in-order traversal of the implicit tree always reflects true sequence position (maintained since Split/Join are purely position-based operations that never reorder existing relative positions),  $\text{xor}(v)$  for any node equals the XOR of exactly the bits in positions spanned by  $T(v)$ ; isolating the range  $[j, k]$  via two Splits therefore makes  $\text{xor}(\text{root}(T_2))$  exactly  $b_j \oplus b_{j+1} \oplus \dots \oplus b_k$ . ■

### Problem 3: Counting Chord Intersections in a Circle

Given  $n$  chords of a circle, count the number of intersecting pairs in  $O(n \log n)$ .

**Algorithmic Intuition.** Label the  $2n$  endpoints by angular position  $0, 1, \dots, 2n - 1$  around the circle. For a chord with endpoint positions  $p < q$  (opening position  $p$ , closing position  $q$ ), two chords  $c_i = (p_i, q_i)$  and  $c_j = (p_j, q_j)$  with  $p_i < p_j$  *cross* iff  $p_i < p_j < q_i < q_j$  — i.e.  $c_j$  opens *during*  $c_i$ 's span and closes *after* it (the "interleaving" configuration; nested or disjoint chords do not cross). Equivalently: at the moment chord  $c_j$  opens, it crosses exactly those currently-open chords whose closing position is  $< q_j$ .

---

**Algorithm 16:** CountChordIntersections(chords, endpoints sorted by angle  $0..2n - 1$ )

---

```

1 Initialize an augmented balanced BST (order-statistics tree)  $S$ , empty, indexed by
  closing-position values, supporting Insert, Delete, and CountLess( $x$ ) = #elements  $< x$ ,
  each in  $O(\log n)$ ;
2 ans  $\leftarrow 0$ ;
3 for pos  $\leftarrow 0$  to  $2n - 1$  do
4   if pos is the opening endpoint of some chord  $c$  with closing position  $q_c$  then
5     ans  $\leftarrow$  ans +  $S$ .CountLess( $q_c$ ) ;    // currently-open chords that will close
        before  $c$  does
6      $S$ .Insert( $q_c$ );
7   else
8     // pos is a closing endpoint of chord  $c$ 
       $S$ .Delete( $q_c$ );
9 return ans;
```

---

**Complexity Analysis.** There are  $2n$  endpoint events; each triggers  $O(1)$  order-statistics-tree operations (Insert, Delete, or CountLess, i.e. a rank query), each  $O(\log n)$  on a balanced augmented BST (size-augmented as in Section 2, Q1). Total:  $O(n \log n)$ .

#### Proof of Correctness.

**Claim 2.3.** Two chords  $c_i$  (opening at  $p_i$ ) and  $c_j$  (opening at  $p_j$ ), with  $p_i < p_j$ , cross iff  $q_i < q_j$  and  $c_i$  is still open when  $c_j$  opens (i.e.  $q_i > p_j$ ).

**Proof Sketch.** Since  $p_i < p_j$ , exactly one of three mutually exclusive geometric configurations holds: (1)  $q_i < p_j$  —  $c_i$  closes before  $c_j$  opens (disjoint arcs, no crossing); (2)  $p_j < q_j < q_i$  —  $c_j$  is entirely nested within  $c_i$  (no crossing, since  $c_i$  both starts before and ends after  $c_j$ ); (3)  $p_j < q_i < q_j$  — the endpoints interleave as  $p_i < p_j < q_i < q_j$ , which is exactly the definition of two chords crossing inside the circle. Case (3) is exactly " $c_i$  open when  $c_j$  opens ( $q_i > p_j$ ) and  $q_i < q_j$ ", proving the claim.

By this claim, when chord  $c_j$  opens, the set of currently-open chords in  $S$  is exactly  $\{c_i : p_i < p_j < q_i\}$  (chords opened but not yet closed), and among these, exactly those with  $q_i < q_j$  cross  $c_j$  — precisely  $S$ .CountLess( $q_j$ ). Summing this over all  $n$  opening events counts every crossing pair exactly once (attributed to the later-opening chord of the pair), giving the true total number of intersections. ■

## Problem 4: Red-Blue Segment Intersections

Given  $n$  red horizontal and  $n$  blue vertical axis-parallel segments, count all red-blue intersections in  $O(n \log n)$ .

**Algorithmic Intuition.** Sweep a vertical line left to right over the  $x$ -axis. A red horizontal segment  $[x_1, x_2] \times \{y\}$  is "active" (intersects the sweep line) for  $x \in [x_1, x_2]$ ; we insert it into an augmented BST keyed by  $y$ -coordinate when the sweep reaches  $x_1$  and remove it at  $x_2$ . A blue vertical segment  $\{x\} \times [y_1, y_2]$ , being a single vertical line, simply queries: "how many currently-active red segments have  $y \in [y_1, y_2]$ ?" — an order-statistics range-count query.

---

### Algorithm 17: CountRedBlueIntersections

---

```

1 Create events: for each red segment, an (INSERT,  $x_1, y$ ) and (DELETE,  $x_2, y$ ) event; for
  each blue segment, a (QUERY,  $x, y_1, y_2$ ) event;
2 Sort all  $3n$  events by  $x$ -coordinate; events with the same  $x$ -coordinate are processed in the
  order INSERT  $\rightarrow$  QUERY  $\rightarrow$  DELETE (the correct convention for closed segments);
3 Initialize augmented order-statistics BST  $S$  (keyed by  $y$ ), supporting Insert, Delete,
  RangeCount( $y_1, y_2$ ) in  $O(\log n)$ ;
4  $\text{ans} \leftarrow 0$ ;
5 foreach event in sorted order do
6   switch event type do
7     case INSERT do  $S.\text{Insert}(y)$ ;
8     case DELETE do  $S.\text{Delete}(y)$ ;
9     case QUERY do  $\text{ans} \leftarrow \text{ans} + S.\text{RangeCount}(y_1, y_2)$ ;
10 return ans;
```

---

where  $\text{RangeCount}(y_1, y_2) = \text{Rank}(y_2) - \text{Rank}(y_1^-)$ , computed via two rank descents ( $O(\log n)$ ) on the size-augmented BST of Section 2, Q1.

**Complexity Analysis.** Sorting  $3n$  events:  $O(n \log n)$ . Each of the  $3n$  events performs one  $O(\log n)$  BST operation. Total:  $O(n \log n)$ .

**Proof of Correctness.** Fix the event convention precisely: events are processed by increasing  $x$ -coordinate and, among events with the same  $x$ -coordinate, in the order INSERT, then QUERY, then DELETE.

*Sweep-line invariant.* At the moment a blue segment's query event at abscissa  $x$  is processed,  $S$  contains exactly the red segments  $r$  with  $x_1^{(r)} \leq x \leq x_2^{(r)}$ . Indeed: a red segment with  $x_1^{(r)} \leq x$  has already been inserted, because its INSERT event has  $x$ -coordinate  $x_1^{(r)} \leq x$  and, in the boundary case  $x_1^{(r)} = x$ , precedes the QUERY by the tie rule; and it has not yet been deleted exactly when  $x_2^{(r)} \geq x$ , because its DELETE event either has  $x$ -coordinate  $x_2^{(r)} > x$  or, in the boundary case  $x_2^{(r)} = x$ , is processed *after* the QUERY by the tie rule. Conversely, a red segment with  $x_1^{(r)} > x$  has not been inserted yet and one with  $x_2^{(r)} < x$  has already been deleted. Thus both endpoints are treated as belonging to the segment, which is precisely the closed-interval semantics  $[x_1, x_2] \times \{y\}$  used in the problem: a red segment whose right endpoint is exactly  $x$  is still active when the blue segment at  $x$  queries, as it must be, since the two segments do intersect at  $(x, y^{(r)})$  when  $y^{(r)} \in [y_1, y_2]$ .  $\text{RangeCount}(y_1, y_2)$  then correctly counts, among these active red segments, exactly those with  $y$ -coordinate in  $[y_1, y_2]$  — precisely the red segments intersecting this blue segment, since

a horizontal segment at height  $y^{(r)} \in [y_1, y_2]$  active at  $x$  intersects the vertical segment  $\{x\} \times [y_1, y_2]$  at the point  $(x, y^{(r)})$ , and conversely. Summing over all blue segments' query events therefore gives the exact total intersection count. ■

## Problem 5: The Josephus Permutation

(a) **Constant  $m$ :  $O(n)$  Algorithm.**

**Intuition.** Use a circular doubly-linked list of the  $n$  people. Repeatedly advance  $m - 1$  pointers and remove the  $m$ -th person, recording the removal order.

---

**Algorithm 18:** Josephus-Constant- $m(n, m)$

---

```

1 Build a circular doubly-linked list  $L = 1 \rightarrow 2 \rightarrow \dots \rightarrow n \rightarrow 1$ ;
2  $\text{cur} \leftarrow L.\text{head}$ ;  $\text{order} \leftarrow []$ ;
3 while  $L$  is non-empty do
4   for  $k \leftarrow 1$  to  $m - 1$  do
5      $\text{cur} \leftarrow \text{cur}.\text{next}$ 
6   append  $\text{cur}.\text{value}$  to  $\text{order}$ ;
7    $\text{nxt} \leftarrow \text{cur}.\text{next}$ ;
8   remove  $\text{cur}$  from  $L$  (splice out in  $O(1)$ );
9    $\text{cur} \leftarrow \text{nxt}$ ;
10 return  $\text{order}$ ;
```

---

**Complexity.** Each of the  $n$  removals advances the pointer  $m - 1$  times, each an  $O(1)$  linked-list step, and removal itself is  $O(1)$ . Since  $m$  is a *constant*, each removal costs  $O(m) = O(1)$ , for a total of  $O(n)$ .

**Correctness.** Immediate from the direct simulation of the problem's definition: the linked list always represents exactly the people remaining, and advancing  $m - 1$  steps then removing the current node is precisely "counting off  $m$  people and removing the  $m$ -th," by construction — an exact, faithful simulation, hence trivially correct by definition equivalence.

(b) **General  $m$ :  $O(n \log n)$  Algorithm.**

**Intuition.** When  $m$  is not constant, direct simulation costs  $\Theta(nm)$  in the worst case. Instead, maintain the  $n$  (eventually shrinking) remaining people in an *order-statistics* augmented BST (size-augmented, Section 2 Q1), supporting  $\text{Select}(\text{rank } k) = \text{the } k\text{-th smallest remaining label}$  in  $O(\log n)$ , and  $\text{Delete}$  in  $O(\log n)$ . Track a current logical "clock position"  $r$  (a rank within the remaining circular structure) and jump directly via modular index arithmetic instead of stepping one-by-one.



---

**Algorithm 19:** Josephus-General- $m(n, m)$ 

---

```
1 Build order-statistics BST  $S$  containing  $1, \dots, n$  (sorted by original position);
2  $r \leftarrow 0$ ; // 0-indexed rank of the "current" starting position
3  $\text{order} \leftarrow []$ ;
4 for  $\text{size} \leftarrow n$  to 1 (step  $-1$ ) do
5    $r \leftarrow (r + m - 1) \bmod \text{size}$ ; // rank, among the size remaining people, of the
   one to remove
6    $x \leftarrow S.\text{Select}(r)$ ; //  $O(\log n)$ : the  $(r + 1)$ -th smallest remaining label
7   append  $x$  to  $\text{order}$ ;
8    $S.\text{Delete}(x)$ ; //  $O(\log n)$ 
   // after deletion, the next count starts immediately after the removed
   slot, which is now rank  $r$  again (mod the new, smaller size)
9 return  $\text{order}$ ;
```

---

**Complexity Analysis.** Each of the  $n$  iterations performs one Select and one Delete, each  $O(\log n)$  on the augmented BST. Total:  $O(n \log n)$ .

**Proof of Correctness.** *Invariant.* Before an iteration in which  $k$  people remain,  $S$  stores exactly the remaining people in their original circular order, and  $r \in \{0, \dots, k - 1\}$  is the 0-indexed rank, among those  $k$ , of the person at whom counting is to start.

*The iteration is correct.* Counting off  $m$  people from rank  $r$  in a circle of  $k$  people visits ranks  $r, r + 1, \dots, r + m - 1$  modulo  $k$ , so the person eliminated is the one of rank  $(r + m - 1) \bmod k$  — exactly the rank the loop computes. Select retrieves the label of that rank in  $O(\log n)$  by the standard order-statistics descent on the size fields (Section 2, Q1), and Delete removes it in  $O(\log n)$ , restoring the size fields along the search path.

*The invariant is restored.* Let  $r' = (r + m - 1) \bmod k$  be the rank just removed. Counting resumes at the successor of the removed person. After the deletion, everyone after the removed slot shifts down one rank, so this successor has rank  $r'$  if  $r' < k - 1$ , and rank 0 if  $r' = k - 1$  (the removed person was last, so the successor is the first one); in both cases its rank in the new circle of  $k - 1$  people is  $r' \bmod (k - 1)$ . Since the next iteration computes  $(r' + m - 1) \bmod (k - 1)$  and  $(r' + m - 1) \equiv ((r' \bmod (k - 1)) + m - 1) \bmod (k - 1)$ , keeping the numeric value  $r = r'$  and decrementing the size is exactly right.

With the invariant established, induction on the number of removals shows that the emitted sequence is exactly the  $(n, m)$ -Josephus permutation. ■

## Problems 6 & 7: Dynamic Sequence Data Structure

Maintain a sequence  $\langle a_1, \dots, a_n \rangle$  supporting, each in  $O(\log n)$ : Insert( $S, i, x$ ), Delete( $S, i$ ), Report( $S, i$ ), Rotate( $S, i, j$ ) (Q6 & Q7), and additionally Multi-add( $S, i, j, x$ ), ReportMin( $S, i, j$ ) (Q7 only). We give one unified augmented data structure supporting the full combined interface (Q6's operations are a strict subset of Q7's).

**Data Structure: Augmented Implicit BST.** Exactly as in Lectures 5 and 6, the sequence is stored in a balanced BST whose *in-order traversal is the sequence itself*: positions are not stored as keys but recovered from the size fields, and a range operation is realised on the  $O(\log n)$  nodes of the two boundary paths together with the  $O(\log n)$  complete subtrees hanging between them,

a whole such subtree being updated in  $O(1)$  through a lazy increment field instead of visiting its nodes. Concretely we balance the tree by random priorities (an implicit treap), which gives height  $O(\log n)$  in expectation, so the bounds below are expected  $O(\log n)$ ; the same augmentation on a red-black or weight-balanced implicit BST gives the same bounds in the worst case.

#### Augmented Fields per node $v$ .

- $\text{val}(v)$ : the stored value at this position.
- $\text{size}(v)$ : subtree size (for positional Select/Split, as before).
- $\text{min}(v)$ : minimum value in  $T(v)$ , satisfying  $\text{min}(v) = \min(\text{val}(v), \text{min}(\text{left}(v)), \text{min}(\text{right}(v)))$ .
- $\text{lazyAdd}(v)$ : a pending additive update to be applied to the *entire* subtree  $T(v)$  but not yet pushed to children (lazy propagation).
- $\text{lazyRev}(v)$ : a boolean flag marking that  $T(v)$ 's left/right children (and their in-order meaning) are pending a swap — used to implement Rotate as a range-reversal in  $O(\log n)$  via lazy propagation, exactly as in a classic "implicit treap with reverse" structure.

**Pushdown.** Before descending into a node  $v$ 's children for any operation, we  $\text{PUSHDOWN}(v)$ : apply  $\text{lazyAdd}(v)$  to  $\text{val}(\text{left}(v)), \text{val}(\text{right}(v))$  (and accumulate into their own  $\text{lazyAdd}$  and  $\text{min}$  fields,  $\text{min}(\text{child}) += \text{lazyAdd}(v)$ ), then swap  $\text{left}(v) \leftrightarrow \text{right}(v)$  if  $\text{lazyRev}(v)$  is set (propagating the reversal flag onto both children, toggled), and finally clear  $\text{lazyAdd}(v), \text{lazyRev}(v)$ . This is  $O(1)$  per node.

**Core Primitives: Split and Join.**  $\text{Split}(T, i)$  and  $\text{Join}(T_1, T_2)$  are the standard positional treap operations (as in Section 2, Q2), each  $O(\log n)$ , each performing PushDown at every node visited along the  $O(\log n)$ -length path before descending, and recomputing size, min bottom-up on the way back up.

#### Operations.

---

##### Algorithm 20: Insert( $S, i, x$ )

---

- 1  $(T_1, T_2) \leftarrow \text{Split}(S, i - 1)$ ;
  - 2  $v \leftarrow \text{new node, } \text{val}(v) = x, \text{size}(v) = 1, \text{min}(v) = x$ ;
  - 3  $S \leftarrow \text{Join}(\text{Join}(T_1, \{v\}), T_2)$ ;
- 

---

##### Algorithm 21: Delete( $S, i$ )

---

- 1  $(T_1, T_{23}) \leftarrow \text{Split}(S, i - 1)$ ;  $(\{v\}, T_3) \leftarrow \text{Split}(T_{23}, 1)$ ;
  - 2  $S \leftarrow \text{Join}(T_1, T_3)$ ;
- 

---

##### Algorithm 22: Report( $S, i$ )

---

- 1 Descend from root, PushDown at every visited node, using  $\text{size}(\text{left}(v))$  to decide whether the  $i$ -th element is in the left subtree, is  $v$  itself, or is in the right subtree (standard order-statistics Select);
  - 2 **return** val of the located node;
-

---

**Algorithm 23:** Rotate( $S, i, j$ ) — reverse the subrange  $[i, j]$

---

```

1 ( $T_1, T_{23}$ )  $\leftarrow$  Split( $S, i - 1$ ); ( $T_2, T_3$ )  $\leftarrow$  Split( $T_{23}, j - i + 1$ );
2 Set lazyRev(root( $T_2$ ))  $\leftarrow$   $\neg$  lazyRev(root( $T_2$ )) ; //  $O(1)$ : mark the whole  $T_2$  for lazy
   reversal
3  $S \leftarrow$  Join(Join( $T_1, T_2$ ),  $T_3$ );
```

---

**Algorithm 24:** Multi-add( $S, i, j, x$ )

---

```

1 ( $T_1, T_{23}$ )  $\leftarrow$  Split( $S, i - 1$ ); ( $T_2, T_3$ )  $\leftarrow$  Split( $T_{23}, j - i + 1$ );
2 val(root( $T_2$ ))  $+= x$ ; min(root( $T_2$ ))  $+= x$ ; lazyAdd(root( $T_2$ ))  $+= x$  ; //  $O(1)$ : tag the
   whole range
3  $S \leftarrow$  Join(Join( $T_1, T_2$ ),  $T_3$ );
```

---

**Algorithm 25:** ReportMin( $S, i, j$ )

---

```

1 ( $T_1, T_{23}$ )  $\leftarrow$  Split( $S, i - 1$ ); ( $T_2, T_3$ )  $\leftarrow$  Split( $T_{23}, j - i + 1$ );
2 ans  $\leftarrow$  min(root( $T_2$ ));
3  $S \leftarrow$  Join(Join( $T_1, T_2$ ),  $T_3$ );
4 return ans;
```

---

*Remark on indexing.* The worked example in the problem statement appears to contain a minor indexing inconsistency. The implementation above follows the natural definition of Rotate( $S, i, j$ ) as reversal of the subrange  $[i, j]$ .

**Complexity Analysis.** Every operation performs  $O(1)$  calls to Split/Join, each  $O(\log n)$  expected (treap height), plus  $O(1)$  additional work at the isolated subtree root. Hence every operation — Insert, Delete, Report, Rotate, Multi-add, ReportMin — runs in  $O(\log n)$  expected time.

**Proof of Correctness.** *Positional consistency.* Since Split/Join operate purely on positions (never on values), the in-order sequence of values in the treap always exactly equals the current logical sequence  $S$ ; this is preserved inductively by every operation, each of which is a composition of Splits (which partition the current in-order sequence at the given position with no reordering) and Joins (concatenation), so the abstract sequence transformation performed (insert one element / delete one element / reverse a contiguous range / add  $x$  to a contiguous range / query the range) matches exactly the semantics defined for each named operation. *Lazy-tag correctness.* We prove by induction on the number of PushDown calls that at every moment, for a node  $v$ , the "true" value/min of  $T(v)$  equals the field currently stored *plus* the sum of all lazyAdd tags on the path from  $v$  up to (but not including) the last ancestor whose lazy tags have already been pushed to  $v$ , and the "true" left-to-right order of  $T(v)$ 's in-order traversal equals its stored structure XORed with the parity of pending lazyRev flags on that same path. Each PushDown exactly discharges one level of pending tags onto the children while preserving this invariant (folding the parent's tag into the child's own lazy field and immediate value/min, and toggling the child's reversal flag), so after PushDown is applied along the entire root-to-node path before any read or structural change at that node (as our Split/Report/etc. always do), all fields read are correct with no pending obligations. Since min( $v$ ) is always correctly recomputed as the min over  $\{\text{val}(v)\} \cup \{\text{min}(\text{children})\}$  immediately after any subtree change (bottom-up, along the  $O(\log n)$ -length modified path), ReportMin returns the true minimum of the isolated range  $T_2$ , and by the positional-consistency argument  $T_2$  exactly corresponds to  $S[i..j]$ . ■

### 3 Greedy Strategy

#### Problem 1: Minimum Mobile Towers

Place the minimum number of towers (radius  $c$ ) along a line of  $n$  houses so every house is within distance  $c$  of some tower.

**Algorithmic Intuition.** Process houses left to right. Whenever we encounter a house not yet covered, greedily place a tower at position (that house's coordinate  $+ c$ ) — the *furthest right* point that still covers this house — since this choice covers this house and maximizes the range of houses covered to the right, never doing worse than any other valid choice for covering this house.

---

#### Algorithm 26: MinTowers(houses[1..n])

---

```

1 Sort houses by coordinate;
2 towers  $\leftarrow []$ ;  $i \leftarrow 1$ ;
3 while  $i \leq n$  do
4    $t \leftarrow \text{houses}[i] + c$ ;           // place a tower to cover houses[i], as far right as
   possible
5   append  $t$  to towers;
6   while  $i \leq n$  and  $\text{houses}[i] \leq t + c$  do
7      $i \leftarrow i + 1$ 
   ;                               // skip all houses now covered by this tower
8 return towers;
```

---

**Complexity Analysis.** Sorting:  $O(n \log n)$ . The main loop's inner while advances  $i$  monotonically from 1 to  $n + 1$  over the whole run (never resets), so the two nested loops together touch each house  $O(1)$  times, giving  $O(n)$  for the scan. Total:  $O(n \log n)$ .

#### Proof of Correctness (Exchange Argument).

**Theorem 3.1.** *The greedy algorithm produces the minimum possible number of towers.*

**Proof Sketch.** Let  $\text{OPT} = (o_1 < o_2 < \dots < o_k)$  be an optimal tower placement and  $\text{GREEDY} = (g_1 < g_2 < \dots < g_{k'})$  the greedy placement. We show by induction that for every  $\ell$ ,  $\text{OPT}$  can be modified (without increasing its tower count) so that its first  $\ell$  towers coincide with  $\text{GREEDY}$ 's first  $\ell$  towers — the standard "greedy stays ahead" argument.

*Base case* ( $\ell = 1$ ). Let  $h_1$  be the leftmost house. It must be covered by some tower of  $\text{OPT}$ , and we may take that tower to be  $o_1$  (the leftmost one, since any tower covering  $h_1$  is at position  $\leq h_1 + c$  and towers to the right of  $o_1$  covering  $h_1$  could be renamed); thus  $|o_1 - h_1| \leq c$ , i.e.  $o_1 \in [h_1 - c, h_1 + c]$ . Greedy sets  $g_1 = h_1 + c$ , the rightmost point that still covers  $h_1$ , so  $g_1 \geq o_1$ . We claim replacing  $o_1$  by  $g_1$  keeps the placement feasible. Let  $v$  be any house covered by  $o_1$ , i.e.  $|v - o_1| \leq c$ . On the right,  $v \leq o_1 + c \leq g_1 + c$  since  $g_1 \geq o_1$ . On the left,  $v \geq h_1 = g_1 - c$ , because  $h_1$  is the leftmost house. Hence  $g_1 - c \leq v \leq g_1 + c$ , so  $v$  is covered by  $g_1$  as well. The substitution is therefore valid and does not change the number of towers.

*Inductive step.* Assume  $\text{OPT}$ 's first  $\ell - 1$  towers have been replaced by  $\text{GREEDY}$ 's without increasing count. Let  $h_\ell$  be the leftmost house not covered by  $\text{GREEDY}$ 's (and, by the modified  $\text{OPT}$ 's) first  $\ell - 1$  towers. By the identical argument as the base case (applied to the sub-problem restricted to houses from  $h_\ell$  onward),  $\text{OPT}$ 's  $\ell$ -th tower  $o_\ell$  can be replaced by  $g_\ell = h_\ell + c$  without losing coverage or increasing the count.

By induction, after  $k' = |\text{GREEDY}|$  steps, all of GREEDY's towers can replace the first  $k'$  towers of OPT without increasing OPT's count, and since GREEDY's towers by construction leave no house uncovered once all  $n$  houses are processed, OPT cannot have used fewer than  $k'$  towers (otherwise the exchanged OPT, with  $< k'$  towers, would fail to cover some house that only the  $k'$ -th greedy step handles — contradiction, since the exchange preserves coverage of everything OPT originally covered). Hence  $|\text{OPT}| \geq |\text{GREEDY}|$ , and since OPT is optimal,  $|\text{OPT}| = |\text{GREEDY}|$ . ■

## Problem 2: Supercomputer + PC Scheduling

$n$  jobs each need  $p_i$  time on a single supercomputer (sequential) followed by  $f_i$  time on a dedicated PC (parallel, unlimited PCs). Minimize the makespan (time all jobs finish on their PCs).

**Algorithmic Intuition.** Since PCs run in parallel with no contention, once a job finishes on the supercomputer it finishes on its PC independently. If the supercomputer processes jobs in order  $\pi(1), \dots, \pi(n)$ , job  $\pi(k)$  finishes the supercomputer stage at time  $S_k = \sum_{i=1}^k p_{\pi(i)}$  and finishes overall at  $S_k + f_{\pi(k)}$ . The makespan is  $\max_k (S_k + f_{\pi(k)})$ . To minimize this maximum, intuitively jobs with a *large* remaining PC time  $f_i$  should be fed to the supercomputer *early* (so their long PC tail has more time to run concurrently with other jobs' supercomputer processing) — i.e. sort by  $f_i$  **descending**.

---

**Algorithm 27:** ScheduleSupercomputer(jobs  $\{(p_i, f_i)\}$ )

---

```

1 Sort jobs in decreasing order of  $f_i$ ;
2  $t \leftarrow 0$ ; makespan  $\leftarrow 0$ ;
3 foreach job  $i$  in sorted order do
4    $t \leftarrow t + p_i$ ;
5   makespan  $\leftarrow \max(\text{makespan}, t + f_i)$ ;
6 return (sorted order, makespan);
```

---

**Complexity Analysis.** Dominated by the sort:  $O(n \log n)$ ; the scan is  $O(n)$ .

### Proof of Correctness (Exchange Argument).

**Theorem 3.2.** *Sorting by  $f_i$  descending minimizes the makespan  $\max_k (S_k + f_{\pi(k)})$ .*

**Proof Sketch.** Let  $\pi$  be any optimal order and suppose there exist adjacent positions  $k, k+1$  in  $\pi$  with  $f_{\pi(k)} < f_{\pi(k+1)}$  (an "out-of-order" adjacent pair relative to the descending- $f$  rule). We show swapping them does not increase the makespan.

Let  $t = S_{k-1}$  (time on the supercomputer just before these two jobs), and let  $a = \pi(k), b = \pi(k+1)$  with  $f_a < f_b$ . Before the swap, job  $a$  finishes at  $t + p_a + f_a$  and job  $b$  finishes at  $t + p_a + p_b + f_b$ . After swapping to order  $b, a$ : job  $b$  finishes at  $t + p_b + f_b$  and job  $a$  finishes at  $t + p_b + p_a + f_a$ .

All other jobs' finish times are unaffected by this local swap (their supercomputer start times, being outside positions  $k, k+1$ , or depending only on the *total*  $p_a + p_b$  processed within these two slots — which is unchanged since both jobs are still processed, just reordered — are identical before and after).

We compare the maximum of the two affected finish times in each case:

$$\text{Before max} = \max(t + p_a + f_a, t + p_a + p_b + f_b), \quad \text{After max} = \max(t + p_b + f_b, t + p_a + p_b + f_a).$$

We bound each term of the "after" maximum by the quantity  $t + p_a + p_b + f_b$ , which is one of the two terms of the "before" maximum: first,  $t + p_b + f_b \leq t + p_a + p_b + f_b$  because  $p_a \geq 0$ ; second,  $t + p_a + p_b + f_a < t + p_a + p_b + f_b$  because  $f_a < f_b$ . Therefore

$$\text{After max} \leq t + p_a + p_b + f_b \leq \text{Before max}.$$

So swapping an adjacent inversion (relative to descending- $f$  order) never increases the overall makespan (since the maximum over all  $n$  finish times can only be affected through these two positions' local maxima, and we just showed that local max does not increase). Repeating this exchange removes all adjacent inversions in finitely many steps (a standard bubble-sort argument bounds this by  $O(n^2)$  swaps, each non-increasing the objective), converging to the fully  $f$ -descending order without ever increasing the makespan from that of the original optimal  $\pi$ . Hence the  $f$ -descending order is itself optimal. ■

### Problem 3: Minimizing Weighted Completion Time

Given jobs with processing times  $t_i$  and weights  $w_i$  on a single machine, order them to minimize  $\sum_i w_i C_i$ .

**Algorithmic Intuition (Smith's Rule).** Intuitively, a job that is both quick ( $t_i$  small) and important ( $w_i$  large) should go first. The precise tiebreaking criterion is the ratio  $w_i/t_i$ : sort in **decreasing** order of  $w_i/t_i$ .

---

**Algorithm 28:** MinWeightedCompletion(jobs  $\{(t_i, w_i)\}$ )

---

- 1 Sort jobs in decreasing order of  $w_i/t_i$ ;
  - 2 **return** sorted order (schedule jobs in this order back-to-back);
- 

**Complexity Analysis.**  $O(n \log n)$  for the sort; computing  $\sum w_i C_i$  afterward is  $O(n)$ .

### Proof of Correctness (Adjacent Exchange Argument).

**Theorem 3.3.** *Ordering jobs by decreasing  $w_i/t_i$  minimizes  $\sum_i w_i C_i$ .*

**Proof Sketch.** Let  $\pi$  be an optimal schedule and suppose jobs  $a, b$  occupy adjacent positions  $k, k+1$  with  $w_a/t_a < w_b/t_b$  (violating the proposed rule). Let  $t = S_{k-1}$  be the total processing time of jobs before this pair. Swapping  $a, b$  changes only  $C_a, C_b$  (all other completion times are determined solely by the total processing time of jobs before them, unaffected by reordering *within* this pair, since  $t_a + t_b$  is preserved).

Before swap ( $a$  then  $b$ ):  $C_a = t + t_a$ ,  $C_b = t + t_a + t_b$ . Contribution:  $w_a(t + t_a) + w_b(t + t_a + t_b)$ .

After swap ( $b$  then  $a$ ):  $C'_b = t + t_b$ ,  $C'_a = t + t_b + t_a$ . Contribution:  $w_b(t + t_b) + w_a(t + t_b + t_a)$ .

Difference (Before – After):

$$\begin{aligned} & [w_a(t + t_a) + w_b(t + t_a + t_b)] - [w_b(t + t_b) + w_a(t + t_b + t_a)] \\ &= w_a t_a + w_b t_a + w_b t_b - w_b t_b - w_a t_b - w_a t_a \\ &= w_b t_a - w_a t_b. \end{aligned}$$

Since  $w_a/t_a < w_b/t_b \iff w_a t_b < w_b t_a$  (as  $t_a, t_b > 0$ ), we get Before – After =  $w_b t_a - w_a t_b > 0$ , i.e. **Before > After**: the swap strictly *decreases* the total weighted completion time. This contradicts optimality of  $\pi$  (which had  $a$  before  $b$ ) unless no such inversion exists. Hence any optimal schedule

has no adjacent pair violating decreasing- $w_i/t_i$  order, i.e. is fully sorted by decreasing  $w_i/t_i$  (a total order with no adjacent inversions must be sorted); since the greedy schedule is exactly this order, it is optimal. ■

## Problem 4: Interval Scheduling

Given  $n$  jobs with start/finish times, find the maximum-size subset of pairwise non-overlapping jobs, in  $O(n \log n)$ .

**Algorithmic Intuition.** Always greedily pick the job with the earliest finish time among those still compatible with previously selected jobs — this leaves maximal remaining room for future selections.

---

**Algorithm 29:** IntervalScheduling(jobs  $\{(s_i, f_i)\}$ )

---

```

1 Sort jobs by finish time  $f_i$  ascending;
2  $R \leftarrow \emptyset$ ; lastFinish  $\leftarrow -\infty$ ;
3 foreach job  $i$  in sorted order do
4   if  $s_i \geq \text{lastFinish}$  then
5      $R \leftarrow R \cup \{i\}$ ; lastFinish  $\leftarrow f_i$ ;
6 return  $R$ ;
```

---

**Complexity Analysis.** Sort:  $O(n \log n)$ . Scan:  $O(n)$ . Total:  $O(n \log n)$ .

## Proof of Correctness ("Greedy Stays Ahead").

**Theorem 3.4.** *IntervalScheduling produces a maximum-cardinality compatible set.*

**Proof Sketch.** Let GREEDY select jobs  $g_1, \dots, g_k$  in order of increasing finish time, and let  $\text{OPT} = o_1, \dots, o_m$  be any optimal solution, also listed in increasing order of finish time. Since GREEDY is a compatible set,  $k \leq m$ ; our goal is to show  $m \leq k$  as well. We first show by induction that  $f(g_r) \leq f(o_r)$  for every  $r = 1, \dots, k$  ("greedy stays ahead" in finish time).

*Base case.*  $g_1$  has the globally earliest finish time among ALL jobs (greedy picks the earliest-finishing job first, unconditionally), so in particular  $f(g_1) \leq f(o_1)$ .

*Inductive step.* Assume  $f(g_r) \leq f(o_r)$ . Since  $o_{r+1}$  starts after  $o_r$  finishes (compatibility of OPT) and  $f(o_r) \geq f(g_r)$ , we have  $s_{o_{r+1}} \geq f(o_r) \geq f(g_r)$ , so  $o_{r+1}$  is compatible with  $g_r$  (starts after  $g_r$ 's finish) and was available to greedy at the step after selecting  $g_r$ . Greedy, being greedy, would then have selected *some* job with finish time  $\leq f(o_{r+1})$  at that step (either  $o_{r+1}$  itself or something finishing even earlier that was available) — precisely  $g_{r+1}$  — so  $f(g_{r+1}) \leq f(o_{r+1})$ .

By induction this holds for all  $r \leq k$ . Now suppose for contradiction  $m > k$ . Then  $o_{k+1}$  exists with  $s_{o_{k+1}} \geq f(o_k) \geq f(g_k)$ , so  $o_{k+1}$  is compatible with  $g_k$  and available to greedy immediately after it picked  $g_k$  — but greedy terminated after  $k$  selections, meaning no further job was available/compatible, contradicting the existence of a compatible, available  $o_{k+1}$ . Hence  $m \leq k$ , and since GREEDY itself is a valid compatible set of size  $k$ , we conclude  $m = k$ : GREEDY is optimal. ■

## Problem 5: Circular (24-Hour) Interval Scheduling

Jobs may wrap past midnight on a 24-hour cyclic processor; maximize the number of accepted (pairwise non-overlapping, in the circular sense) jobs, in polynomial time.

**Model and notation.** Let  $D = 24$  be the length of the day and identify times with points of the circle  $\mathbb{R}/D\mathbb{Z}$ . Job  $i$  is given by  $(s_i, e_i)$  and occupies the half-open arc traversed clockwise from  $s_i$  to  $e_i$ ,

$$A_i = \{ (s_i + x) \bmod D : 0 \leq x < \text{len}_i \}, \quad \text{len}_i = (e_i - s_i) \bmod D,$$

so a job with  $s_i > e_i$  automatically wraps past midnight. We assume  $0 < \text{len}_i < D$  (no empty job and no job occupying the whole day). Two jobs are *compatible* iff  $A_i \cap A_j = \emptyset$ ; a feasible schedule is a set of pairwise compatible jobs; OPT denotes a feasible schedule of maximum cardinality.

**Algorithmic Intuition.** On a line, interval scheduling is easy because there is a distinguished point (namely  $-\infty$ ) that no job covers. On a circle no such point is given — but any feasible schedule that selects a job  $a$  leaves the whole arc  $A_a$  free of the other selected jobs, so the circle may be cut anywhere inside  $A_a$  and, in particular, at the point  $e_a$  where  $A_a$  ends. Cutting there and unrolling turns every job compatible with  $a$  into an ordinary non-wrapping interval of a line segment, so the residual problem is plain linear interval scheduling and earliest-finish-time greedy solves it. Since the anchor  $a$  is unknown, we try all  $n$  jobs as anchors and keep the best trial. The unrolling is not merely a change of viewpoint: the endpoints of the surviving jobs must actually be recomputed in the coordinate system of the cut, and the greedy must use those transformed finish times.

**The anchor-dependent transformation.** Fix an anchor  $a$ . Cut the circle at  $e_a$  and use the coordinate

$$\varphi(p) = (p - e_a) \bmod D \in [0, D),$$

which maps the free arc  $C_a$  (the complement of  $A_a$ , of length  $\text{gap}_a = D - \text{len}_a$ , starting at  $e_a$ ) bijectively onto  $[0, \text{gap}_a)$ . Every job  $j \neq a$  is transformed into the interval

$$I_j = [s'_j, e'_j), \quad s'_j = \varphi(s_j) = (s_j - e_a) \bmod D, \quad e'_j = s'_j + \text{len}_j,$$

where the finish coordinate is deliberately *not* reduced modulo  $D$ : the test  $e'_j \leq \text{gap}_a$  then both decides whether  $j$  is compatible with  $a$  and certifies that  $I_j$  does not cross the cut (Lemma 3.5).

---

**Algorithm 30:** CircularIntervalScheduling(jobs  $\{(s_i, e_i)\}_{i=1}^n$ , day length  $D$ )

---

```

1 foreach job  $i$  do
2    $\text{len}_i \leftarrow (e_i - s_i) \bmod D$ 
3  $E[1..n] \leftarrow$  the jobs sorted by finish time  $e_i$  ascending ; // one global sort,  $O(n \log n)$ 
4  $\text{best} \leftarrow \emptyset$ ;
5 foreach job  $a$  do
6    $R \leftarrow \text{AnchorTrial}(a)$ ;
7   if  $|R| > |\text{best}|$  then  $\text{best} \leftarrow R$ ;
8 return best;
```

---



---

**Algorithm 31:** AnchorTrial( $a$ ) — best schedule among those containing  $a$ 


---

```

1  $\text{gap}_a \leftarrow D - \text{len}_a$  ; // length of the free arc, from  $e_a$  clockwise to  $s_a$ 
2  $p \leftarrow$  smallest index with  $e_{E[p]} \geq e_a$ , or 1 if no such index (binary search,  $O(\log n)$ );
3  $L \leftarrow$  empty list ; // transformed intervals, already in increasing  $e'$  order
4 for  $q \leftarrow 0$  to  $n - 1$  do
5    $j \leftarrow E[(p - 1 + q) \bmod n + 1]$  ; // cyclic scan of  $E$  starting at  $p$ 
6   if  $j = a$  then continue;
7    $s'_j \leftarrow (s_j - e_a) \bmod D$ ;  $e'_j \leftarrow s'_j + \text{len}_j$ ;
8   if  $e'_j \leq \text{gap}_a$  then append  $(j, s'_j, e'_j)$  to  $L$  ; //  $j$  is compatible with  $a$ 
9   ;
10  $S \leftarrow \emptyset$ ;  $\text{last} \leftarrow 0$ ;
11 foreach  $(j, s'_j, e'_j) \in L$  in order do
12   if  $s'_j \geq \text{last}$  then
13      $S \leftarrow S \cup \{j\}$ ;  $\text{last} \leftarrow e'_j$ ;
14 return  $\{a\} \cup S$ ;

```

---

**Worked example.** Take  $D = 24$  and the jobs  $a_1 = (22, 2)$  (wrapping),  $a_2 = (3, 7)$ ,  $a_3 = (6, 10)$ ,  $a_4 = (11, 20)$ . With anchor  $a = a_1$  we get  $\text{len}_{a_1} = 4$ ,  $\text{gap}_a = 20$  and the cut at  $e_a = 2$ . The transformations are  $a_2 \mapsto [1, 5)$ ,  $a_3 \mapsto [4, 8)$ ,  $a_4 \mapsto [9, 18)$ , all with  $e' \leq 20$ , so all three survive the filter; sorted by  $e'$  they are  $[1, 5)$ ,  $[4, 8)$ ,  $[9, 18)$ . The greedy takes  $[1, 5)$ , rejects  $[4, 8)$  (as  $4 < 5$ ), takes  $[9, 18)$ , giving  $R_{a_1} = \{a_1, a_2, a_4\}$  of size 3, which is optimal here.

**Complexity Analysis.** Computing all lengths is  $O(n)$  and the single global sort is  $O(n \log n)$ . One call of AnchorTrial performs a binary search ( $O(\log n)$ ), one cyclic pass over the  $n$  jobs doing  $O(1)$  arithmetic each, and one pass of the greedy over  $L$ :  $O(n)$  in total, *with no sorting inside the loop* — this is legitimate because the transformed finish order is a rotation of the global finish order (Lemma 3.7). With  $n$  anchors the total is

$$O(n \log n) + n \cdot O(n) = O(n^2)$$

time and  $O(n)$  space. (If instead one sorted the transformed intervals from scratch inside each trial, the bound would be  $O(n^2 \log n)$ ; the rotation argument is exactly what avoids this.)

### Proof of Correctness.

**Lemma 3.5** (Filter and unrolling). *Fix an anchor  $a$  and a job  $j \neq a$ . Then  $j$  is compatible with  $a$  if and only if  $e'_j \leq \text{gap}_a$ . Moreover, in that case  $\varphi(A_j) = [s'_j, e'_j) \subseteq [0, \text{gap}_a)$ , i.e. the transformed interval is a genuine non-wrapping interval of the line that lies inside the free segment.*

*Proof.* The complement of  $A_a$  is the arc  $C_a = \{(e_a + x) \bmod D : 0 \leq x < \text{gap}_a\}$ , and  $\varphi$  is a bijection of the circle onto  $[0, D)$  with  $\varphi(C_a) = [0, \text{gap}_a)$ . Since  $A_j$  and  $A_a$  are arcs,  $A_j \cap A_a = \emptyset$  iff  $A_j \subseteq C_a$ . Now

$$\varphi(A_j) = \{(s'_j + x) \bmod D : 0 \leq x < \text{len}_j\}.$$

If  $e'_j = s'_j + \text{len}_j \leq \text{gap}_a (\leq D)$ , no reduction modulo  $D$  occurs, so  $\varphi(A_j) = [s'_j, e'_j) \subseteq [0, \text{gap}_a) = \varphi(C_a)$  and hence  $A_j \subseteq C_a$ . Conversely, assume  $A_j \subseteq C_a$ . Then  $s_j \in C_a$  gives  $s'_j < \text{gap}_a$ ; and if we had  $e'_j > \text{gap}_a$ , the value  $x = \text{gap}_a - s'_j$  would satisfy  $0 \leq x < \text{len}_j$ , so the point of  $A_j$  at offset  $x$  would map to  $\text{gap}_a \notin \varphi(C_a)$ , contradicting  $A_j \subseteq C_a$ . Hence  $e'_j \leq \text{gap}_a$ .  $\square$

**Lemma 3.6** (Compatibility is preserved). *Let  $j, k$  be two jobs compatible with  $a$ . Then  $A_j \cap A_k = \emptyset$  if and only if  $[s'_j, e'_j] \cap [s'_k, e'_k] = \emptyset$ .*

*Proof.* By Lemma 3.5,  $\varphi(A_j) = [s'_j, e'_j]$  and  $\varphi(A_k) = [s'_k, e'_k]$ . As  $\varphi$  is a bijection,  $A_j \cap A_k = \emptyset$  iff  $\varphi(A_j) \cap \varphi(A_k) = \emptyset$ .  $\square$

**Lemma 3.7** (The list  $L$  is sorted by transformed finish time). *For every job  $j$  retained by the filter,  $e'_j = (e_j - e_a) \bmod D$ . Consequently the cyclic scan of  $E$  starting at index  $p$  enumerates the retained jobs in non-decreasing order of  $e'_j$ .*

*Proof.* For a retained job,  $e'_j = s'_j + \text{len}_j \leq \text{gap}_a < D$ , so  $(e_j - e_a) \bmod D = (s_j + \text{len}_j - e_a) \bmod D = (s'_j + \text{len}_j) \bmod D = e'_j$ . The map  $e \mapsto (e - e_a) \bmod D$  is strictly increasing on  $[e_a, D)$  and on  $[0, e_a)$ , and sends the former range below the latter; therefore listing the jobs by increasing  $e_j$  starting from the first index  $p$  with  $e_{E[p]} \geq e_a$  and wrapping around lists them by non-decreasing  $(e_j - e_a) \bmod D$ . Dropping the jobs rejected by the filter preserves this order.  $\square$

**Lemma 3.8** (Each trial is feasible and is optimal among schedules containing its anchor). *Let  $\text{lin}(a)$  be the maximum number of pairwise compatible jobs in  $J'_a := \{j \neq a : A_j \cap A_a = \emptyset\}$ . Then  $\text{AnchorTrial}(a)$  returns a feasible schedule  $R_a$  containing  $a$  with  $|R_a| = 1 + \text{lin}(a)$ , and every feasible schedule containing  $a$  has at most  $1 + \text{lin}(a)$  jobs.*

*Proof.* By Lemma 3.5,  $L$  contains exactly the jobs of  $J'_a$ , each as a genuine interval of the line, and by Lemma 3.7  $L$  is sorted by finish coordinate. The final loop is therefore verbatim the earliest-finish-time greedy of Problem 4 run on the linear instance  $\{I_j : j \in J'_a\}$ , and by the theorem proved there it returns a maximum-cardinality set  $S$  of pairwise disjoint intervals; by Lemma 3.6, pairwise disjointness of the intervals is the same as pairwise circular compatibility of the corresponding jobs, so  $|S| = \text{lin}(a)$  and  $S$  is a set of pairwise compatible jobs. Each  $j \in S$  lies in  $J'_a$  and is thus compatible with  $a$ , so  $R_a = \{a\} \cup S$  is feasible with  $|R_a| = 1 + \text{lin}(a)$ . Finally, if  $F$  is any feasible schedule containing  $a$ , then  $F \setminus \{a\} \subseteq J'_a$  is pairwise compatible, so  $|F| - 1 \leq \text{lin}(a)$ .  $\square$

**Theorem 3.9.** *CircularIntervalScheduling returns a maximum-cardinality feasible circular schedule.*

*Proof. Feasibility.* By Lemma 3.8 every candidate  $R_a$  is feasible, and the algorithm returns one of them (or  $\emptyset$  if there are no jobs at all); hence  $|\text{best}| \leq |\text{OPT}|$ .

*Case coverage and optimality.* If there are no jobs,  $\text{OPT} = \emptyset$  and the algorithm returns  $\emptyset$ . Otherwise, if  $\text{OPT} = \emptyset$  the bound is trivial, so assume  $\text{OPT} \neq \emptyset$  and pick any  $a^* \in \text{OPT}$ . The algorithm tries every job as an anchor, so in particular it executes  $\text{AnchorTrial}(a^*)$ . Since  $\text{OPT}$  is feasible and contains  $a^*$ , Lemma 3.8 gives  $|\text{OPT}| \leq 1 + \text{lin}(a^*) = |R_{a^*}|$ . As  $\text{best}$  is a candidate of maximum size,  $|\text{best}| \geq |R_{a^*}| \geq |\text{OPT}|$ . Combined with feasibility,  $|\text{best}| = |\text{OPT}|$ .  $\square$

Three points deserve emphasis, since they are where circularity could go wrong. (i) The jobs *crossing the cut* are precisely the jobs meeting  $A_a$ : a job containing the cut point  $e_a$  in the interior of its arc also contains points immediately preceding  $e_a$ , which belong to  $A_a$ ; such jobs are exactly the ones rejected by the test  $e'_j \leq \text{gap}_a$  (Lemma 3.5), and they may legitimately be discarded because no schedule containing  $a$  can use them. (ii) After the cut, compatibility means ordinary disjointness of the transformed intervals (Lemma 3.6), so the residual instance really is a linear one and Problem 4's greedy applies unchanged. (iii) The greedy uses the *transformed* finish times, obtained in the coordinate system of the current anchor, not the original circular ones.

## Problem 6: Money-Laundering Interval-Point Matching

Match each of  $n$  account events  $x_1, \dots, x_n$  to a distinct suspicious transaction (given by intervals  $(t_j - e_j, t_j + e_j)$ ) such that  $x_i$  is assigned to a transaction whose interval contains  $x_i$ , in  $O(n^2)$ .

**Algorithmic Intuition.** This is a bipartite matching between points  $\{x_i\}$  and intervals  $\{(t_j - e_j, t_j + e_j)\}$ , where a point may match an interval iff the point lies inside it. The greedy strategy for interval-point matching: process account-event points in increasing order; for the current (smallest unmatched) point, among all *available* (unmatched) intervals that contain it, match it to the one with the smallest *right endpoint* (earliest deadline) — this is the interval “most urgently needing” a point, since any later point is less likely to still fit inside it.

---

**Algorithm 32:** MatchAccountEvents( $\{x_i\}, \{(t_j - e_j, t_j + e_j)\}$ )

---

```

1 Sort the account events  $x_1 < \dots < x_n$ ;
2 matched  $\leftarrow$  empty map;
3 unmatchedIntervals  $\leftarrow$  all  $n$  transaction intervals;
4 foreach  $x_i$  in increasing order do
5    $C \leftarrow \{ \text{intervals in unmatchedIntervals containing } x_i \}$ ;
6   if  $C = \emptyset$  then
7     return "No valid association exists"
8    $j^* \leftarrow \arg \min_{j \in C} (t_j + e_j)$  ; // smallest right endpoint / earliest deadline
9   matched[ $x_i$ ]  $\leftarrow j^*$ ; remove  $j^*$  from unmatchedIntervals;
10 return matched;
```

---

**Complexity Analysis.** Sorting the  $n$  points:  $O(n \log n)$ . For each of the  $n$  points, scanning all remaining intervals to find  $C$  and the minimum right endpoint takes  $O(n)$  in the straightforward implementation. Total:  $O(n^2)$ , matching the target (an  $O(n \log n)$  refinement is possible using a sweep with a min-heap keyed by right endpoint over currently “active” intervals, but  $O(n^2)$  suffices here and is simpler to state/prove).

### Proof of Correctness (Exchange Argument).

**Theorem 3.10.** *If a valid perfect matching between the  $n$  points and  $n$  intervals exists, the greedy algorithm finds one (and correctly reports infeasibility otherwise).*

**Proof Sketch.** We follow the standard greedy template: isolate the greedy choice, exchange it into an arbitrary optimal (here: an arbitrary valid) solution, reduce to the residual subproblem, and induct on  $n$ . Write  $\text{left}(I), \text{right}(I)$  for the endpoints of an interval  $I$ , so  $x \in I$  means  $\text{left}(I) < x < \text{right}(I)$ . The induction is on the number  $n$  of points, the base case  $n = 0$  being vacuous.

1. *The greedy choice.* With the points sorted as  $x_1 < \dots < x_n$ , let  $C = \{I : x_1 \in I\}$  be the intervals containing the smallest point. Greedy matches  $x_1$  to  $I^* = \arg \min_{I \in C} \text{right}(I)$ .

2. *An arbitrary valid matching.* Suppose some valid matching  $M$  of all  $n$  points to the  $n$  intervals exists (if  $C = \emptyset$  then none exists, since  $x_1$  must be matched to an interval containing it, and greedy correctly reports failure; so assume  $C \neq \emptyset$ ). Since  $M$  is a bijection between the points and the intervals, it matches  $x_1$  to some  $I = M(x_1) \in C$ , and it matches  $I^*$  to some point  $x_k$ ; if  $I = I^*$  there is nothing to prove, so assume  $I \neq I^*$ , whence  $k \neq 1$  and therefore  $x_k > x_1$ .

3. *The exchange.* Let  $M'$  agree with  $M$  everywhere except that  $x_1 \mapsto I^*$  and  $x_k \mapsto I$ . Then  $M'$  is again a bijection, and it is valid:  $x_1 \in I^*$  because  $I^* \in C$ , and  $x_k \in I$  because

$$\text{left}(I) < x_1 < x_k \quad \text{and} \quad x_k < \text{right}(I^*) \leq \text{right}(I),$$

where  $\text{left}(I) < x_1$  holds since  $x_1 \in I$ ,  $x_1 < x_k$  since  $x_1$  is the smallest point,  $x_k < \text{right}(I^*)$  since  $M$  matched  $x_k$  into  $I^*$ , and  $\text{right}(I^*) \leq \text{right}(I)$  by the greedy choice of  $I^*$  as the interval of  $C$  with smallest right endpoint (and  $I \in C$ ). Thus a valid matching exists that agrees with greedy's first choice: the greedy choice is safe.

4. *Reduction to the residual subproblem.* Deleting the pair  $(x_1, I^*)$  from  $M'$  leaves a valid matching of the  $n - 1$  points  $x_2 < \dots < x_n$  to the  $n - 1$  intervals other than  $I^*$  — which is exactly the instance greedy faces after its first iteration.

5. *Induction.* By the inductive hypothesis applied to that residual instance, greedy run on it produces a valid matching of the remaining points; prepending the pair  $(x_1, I^*)$  gives a valid matching of the whole instance, which is precisely the output of greedy on the original instance. Conversely, if greedy reports failure, it does so at some point  $x_i$  for which the residual instance at that stage has no interval containing  $x_i$ ; by steps 2–4 applied  $i - 1$  times, the existence of a valid matching of the original instance would imply the existence of one for that residual instance, which is impossible. Hence greedy finds a valid matching whenever one exists, and correctly reports infeasibility otherwise. ■

## Problem 7: Time-Varying Minimum Spanning Tree

Each edge  $e$  has weight  $a_e t + b_e$  (linear in time  $t$ ). Compute, for a query interval  $[t_1, t_2]$ , the MST with least total weight over that interval, in  $O(m \log n)$ .

**Reading of the Problem Statement.** We are asked for the spanning tree of least total weight over the time interval  $[t_1, t_2]$ : among all spanning trees  $T$  of  $G$  and all times  $t \in [t_1, t_2]$ , find a pair  $(T, t)$  minimising

$$w_T(t) = \sum_{e \in T} (a_e t + b_e),$$

i.e. compute  $\min_{t \in [t_1, t_2]} \text{mst}(t)$ , where  $\text{mst}(t)$  denotes the weight of a minimum spanning tree of  $G$  under the fixed weights  $w_e(t) = a_e t + b_e$ , together with a tree and a time attaining it.

**Algorithmic Intuition.** For each *fixed* spanning tree  $T$ , the total weight

$$w_T(t) = \sum_{e \in T} (a_e t + b_e) = \left( \sum_{e \in T} a_e \right) t + \sum_{e \in T} b_e$$

is an *affine* function of  $t$ , and an affine function on a closed interval attains its minimum at one of the two endpoints. Since this holds for every tree simultaneously, the joint minimum over trees *and* over  $t \in [t_1, t_2]$  is attained at  $t = t_1$  or at  $t = t_2$ . So we need only compute an ordinary MST at these two times — a fixed-weight problem solvable by Kruskal's algorithm — and keep the lighter of the two.

---

**Algorithm 33:** TimeVaryingMST(graph  $G$ , edge params  $\{(a_e, b_e)\}$ , query  $[t_1, t_2]$ )

---

```

1  $T_1 \leftarrow$  MST of  $G$  under the fixed weights  $w_e(t_1) = a_e t_1 + b_e$ , by Kruskal's algorithm;
2  $T_2 \leftarrow$  MST of  $G$  under the fixed weights  $w_e(t_2) = a_e t_2 + b_e$ , by Kruskal's algorithm;
3  $W_1 \leftarrow \sum_{e \in T_1} w_e(t_1)$ ;  $W_2 \leftarrow \sum_{e \in T_2} w_e(t_2)$ ;
4 if  $W_1 \leq W_2$  then
5   | return  $(T_1, t_1, W_1)$ ; // lightest tree over the whole interval, attained at
   |  $t_1$ 
6 else
7   | return  $(T_2, t_2, W_2)$ ; // lightest tree over the whole interval, attained at
   |  $t_2$ 

```

---

**Complexity Analysis.** Evaluating the  $m$  weights  $w_e(t_1)$  takes  $O(m)$ ; one Kruskal run sorts the  $m$  edges and performs  $O(m)$  union-find operations, i.e.  $O(m \log m + m\alpha(n)) = O(m \log n)$  (as  $m \leq n^2$ ,  $\log m = O(\log n)$ ). The same for  $t_2$ . Summing the weights of the two trees is  $O(n)$ . Total:  $O(m \log n)$ .

**Proof of Correctness.**

**Lemma 3.11** (The optimum over the interval sits at an endpoint). *For every spanning tree  $T$  of  $G$  and every  $t \in [t_1, t_2]$ ,*

$$w_T(t) \geq \min(\text{mst}(t_1), \text{mst}(t_2)),$$

where  $\text{mst}(t)$  denotes the weight of a minimum spanning tree of  $G$  under the fixed weights  $w_e(t)$ .

**Proof Sketch.** Fix  $T$ . The function  $w_T(t) = A_T t + B_T$  with  $A_T = \sum_{e \in T} a_e$ ,  $B_T = \sum_{e \in T} b_e$  is affine in  $t$ . Writing  $t = \lambda t_1 + (1 - \lambda)t_2$  with  $\lambda \in [0, 1]$  (possible for every  $t \in [t_1, t_2]$ ), affinity gives

$$w_T(t) = \lambda w_T(t_1) + (1 - \lambda) w_T(t_2) \geq \min(w_T(t_1), w_T(t_2)),$$

since a convex combination of two numbers is at least their minimum. Finally  $w_T(t_i) \geq \text{mst}(t_i)$  for  $i = 1, 2$ , because  $\text{mst}(t_i)$  is by definition the least weight of a spanning tree at time  $t_i$ . Combining the two displays proves the claim.

The lemma says that no pair  $(T, t)$  with  $t$  strictly inside the interval can beat both endpoints, i.e.

$$\min_{t \in [t_1, t_2]} \text{mst}(t) = \min(\text{mst}(t_1), \text{mst}(t_2)).$$

(The inequality " $\geq$ " is exactly the lemma, and " $\leq$ " is trivial because  $t_1, t_2 \in [t_1, t_2]$ .) By the correctness of Kruskal's algorithm on a fixed-weight graph,  $T_1$  and  $T_2$  are minimum spanning trees at  $t_1$  and  $t_2$ , so  $W_1 = \text{mst}(t_1)$  and  $W_2 = \text{mst}(t_2)$ ; the algorithm returns the smaller of the two together with the tree and the time attaining it, which by the display above is a pair of globally least total weight over  $[t_1, t_2]$ . ■

**Remark (optional).** The argument uses nothing beyond the affinity of each tree's weight in  $t$  and the correctness of Kruskal's algorithm at a fixed time. The related *parametric* problem — listing a minimum spanning tree for every  $t \in [t_1, t_2]$ , i.e. all breakpoints of  $\text{mst}(\cdot)$  — is a different and harder problem, and we make no complexity claim about it here.

## Problem 8: Maximum Bottleneck Capacity Path

Given a directed graph with edge capacities  $c(e) > 0$ , find the  $s$ - $t$  path maximizing the minimum edge capacity along it.

**Algorithmic Intuition.** Directly mirror Dijkstra's algorithm, but replace "sum edges, minimize total" with "take the min edge along a path, maximize that min" — a max-priority-queue variant where we greedily expand the vertex with the currently-best-known bottleneck value, and relax by taking min instead of +.

---

**Algorithm 34:** MaxBottleneckPath( $G, s, t$ )

---

```

1 foreach  $v \in V$  do
2    $\text{bottleneck}[v] \leftarrow -\infty$ 
3  $\text{bottleneck}[s] \leftarrow +\infty$ ;
4  $Q \leftarrow$  max-priority queue of all vertices, keyed by  $\text{bottleneck}[\cdot]$ ;
5 while  $Q$  non-empty do
6    $u \leftarrow Q.\text{ExtractMax}()$ ;
7   foreach edge  $(u, v)$  with capacity  $c(u, v)$  do
8      $\text{cand} \leftarrow \min(\text{bottleneck}[u], c(u, v))$ ;
9     if  $\text{cand} > \text{bottleneck}[v]$  then
10       $\text{bottleneck}[v] \leftarrow \text{cand}$ ;  $Q.\text{DecreaseKey-ToIncrease}(v, \text{cand})$ ; store  $\text{pred}[v] \leftarrow u$ ;
11 Reconstruct path  $s \rightarrow t$  via  $\text{pred}$  pointers;
12 return path,  $\text{bottleneck}[t]$ ;

```

---

**Complexity Analysis.** Structurally identical to Dijkstra's algorithm (same extract-max / relax-and-update loop, same number of heap operations:  $n$  ExtractMax and  $m$  potential key updates), so with a binary heap this is  $O((n + m) \log n)$ , or  $O(m + n \log n)$  with a Fibonacci heap.

**Proof of Correctness.** For a path  $P$  let  $\text{bn}(P) = \min_{e \in P} c(e)$  be its bottleneck (with  $\text{bn}(P) = +\infty$  for the empty path), and let

$$\beta(v) = \max\{\text{bn}(P) : P \text{ an } s\text{-}v \text{ path}\}$$

be the optimum we want ( $\beta(v) = -\infty$  if  $v$  is unreachable,  $\beta(s) = +\infty$ ).

**Lemma 3.12.** *Let  $S$  denote the set of vertices already extracted. Then at all times:*

- (a) *for every  $v$ ,  $\text{bottleneck}[v] \leq \beta(v)$ , and  $\text{bottleneck}[v]$  is realised by an actual  $s$ - $v$  path;*
- (b) *for every  $v$ ,  $\text{bottleneck}[v] \geq \text{bn}(P)$  for every  $s$ - $v$  path  $P$  all of whose vertices except  $v$  lie in  $S$ ;*
- (c) *every  $u \in S$  satisfies  $\text{bottleneck}[u] = \beta(u)$ .*

*Proof.* (a) A value  $\text{bottleneck}[v] > -\infty$  is only ever set as  $\min(\text{bottleneck}[u], c(u, v))$  for an edge  $(u, v)$ , so by induction on the assignments it equals  $\text{bn}(P)$  for the concrete path  $P$  obtained by following the  $\text{pred}$  pointers; hence it never exceeds the maximum  $\beta(v)$ .

(b) Induction on  $|S|$ . Initially  $S = \emptyset$  and the only such path is the empty path at  $v = s$ , with  $\text{bottleneck}[s] = +\infty$ . When a vertex  $u$  is extracted, the algorithm relaxes every edge  $(u, v)$ , setting  $\text{bottleneck}[v] \geq \min(\text{bottleneck}[u], c(u, v))$ . Any  $s$ - $v$  path whose internal vertices lie in  $S \cup \{u\}$

either has all internal vertices in  $S$  (covered by the inductive hypothesis, and  $\text{bottleneck}[v]$  never decreases) or has  $u$  as its last internal vertex, in which case it is a path  $P'$  to  $u$  with internal vertices in  $S$  followed by the edge  $(u, v)$ ; then  $\text{bn}(P) = \min(\text{bn}(P'), c(u, v)) \leq \min(\text{bottleneck}[u], c(u, v)) \leq \text{bottleneck}[v]$ , using the inductive hypothesis at  $u$ .

(c) Induction on the extraction order. The first extracted vertex is  $s$  (its key  $+\infty$  is maximum), and  $\text{bottleneck}[s] = +\infty = \beta(s)$ . Suppose (c) holds for all vertices extracted before  $u$ , and let  $u$  be extracted now. By (a) it suffices to show  $\text{bottleneck}[u] \geq \beta(u)$ , i.e.  $\text{bottleneck}[u] \geq \text{bn}(P)$  for every  $s$ - $u$  path  $P$ . Fix such a  $P$  and let  $(x, y)$  be its first edge with  $x \in S$  and  $y \notin S$  (it exists because  $s \in S$  and  $u \notin S$ ; possibly  $y = u$ ). The prefix of  $P$  up to  $y$  has all internal vertices in  $S$ , so by (b)  $\text{bottleneck}[y] \geq \text{bn}(P[s..y]) \geq \text{bn}(P)$ , the last step because a prefix's bottleneck is at least the whole path's bottleneck (a minimum over fewer edges). Since  $u$  was extracted as the vertex of maximum key among those not in  $S$ , and  $y \notin S$ , we get  $\text{bottleneck}[u] \geq \text{bottleneck}[y] \geq \text{bn}(P)$ , as required.  $\square$

By part (c) of the lemma, when  $t$  is extracted  $\text{bottleneck}[t] = \beta(t)$ , and by part (a) the pred pointers reconstruct an actual  $s$ - $t$  path whose bottleneck equals this value. Hence the algorithm returns a maximum-bottleneck  $s$ - $t$  path (and  $\text{bottleneck}[t] = -\infty$  exactly when  $t$  is unreachable).  $\blacksquare$

## Problem 9: Spanning Tree with Exactly $k$ Red Edges

Given a graph with edges colored red/blue and a target  $k$ , determine whether a spanning tree with exactly  $k$  red edges exists, and construct one.

**Algorithmic Intuition.** Use two extremal Kruskal-style runs to find the achievable *range* of red-edge counts over all spanning trees, then use the elementary cycle-property exchange for spanning trees to show every integer in that range is achievable, constructing the target tree by a sequence of single-edge swaps, each of which raises the red count by exactly one.

---

### Algorithm 35: ExactKRedSpanningTree( $G, k$ )

---

```

1  $T_{\max} \leftarrow$  Kruskal's algorithm preferring red edges on ties / processing all red edges before
   all blue edges (subject to maintaining a forest via union-find): yields a spanning tree with
   the maximum possible number of red edges,  $r_{\max}$ ;
2  $T_{\min} \leftarrow$  Kruskal's algorithm processing all blue edges before all red edges: yields a
   spanning tree with the minimum possible number of red edges,  $r_{\min}$ ;
3 if  $k < r_{\min}$  or  $k > r_{\max}$  then
4   return "No such spanning tree exists"
5  $T \leftarrow T_{\min}$ ;  $r \leftarrow r_{\min}$ ;
6 while  $r < k$  do
7   // Find a red non-tree edge whose addition creates a cycle containing a
   // blue tree edge; swap to increase red count by exactly 1
8   foreach red edge  $e' \notin T$  do
9     Find the (unique) cycle formed by  $T \cup \{e'\}$  (via a tree path query,  $O(n)$ );
10    if this cycle contains a blue tree edge  $e_{\text{blue}}$  then
11       $T \leftarrow (T \setminus \{e_{\text{blue}}\}) \cup \{e'\}$ ;  $r \leftarrow r + 1$ ; break;
11 return  $T$ ;
```

---

**Complexity Analysis.** Building  $T_{\max}$  and  $T_{\min}$  costs two Kruskal-style runs,  $O(m \log m) = O(m \log n)$  each (only a stable partition of the edges by colour is needed, plus union-find). The exchange loop runs at most  $k - r_{\min} \leq n - 1$  times. In a straightforward implementation, one iteration scans the  $O(m)$  red non-tree edges and, for each, finds the fundamental cycle by an  $O(n)$  traversal of the current tree, i.e.  $O(mn)$  per exchange and  $O(mn^2)$  in total; this dominates the two Kruskal runs, so the overall bound is  $O(mn^2)$ . Optionally, if the tree-path query is implemented with a Link-Cut Tree (path query and update in  $O(\log n)$ ), one exchange costs  $O(m \log n)$  and the exchange phase costs  $O(mn \log n)$ . We claim no bound better than these.

**Proof of Correctness.** Let  $R$  and  $B$  denote the red and the blue edge sets, let  $\rho(T) = |T \cap R|$  for a spanning tree  $T$ , and let  $c(R)$ ,  $c(B)$  be the numbers of connected components of the spanning subgraphs  $(V, R)$  and  $(V, B)$ . We assume, as the problem does, that  $G$  is connected, and we use only two elementary facts proved with Kruskal's algorithm:

- (i) **Cycle property / exchange.** If  $T$  is a spanning tree and  $e \notin T$ , then  $T \cup \{e\}$  contains exactly one cycle  $C_T(e)$ , and for every  $f \in C_T(e)$  with  $f \neq e$  the set  $(T \setminus \{f\}) \cup \{e\}$  is again a spanning tree.
- (ii) **Forest extension.** Every forest of a connected graph  $G$  extends to a spanning tree of  $G$ , and a forest inside a spanning subgraph with  $c$  components has at most  $n - c$  edges, with equality exactly when it is a maximal forest of that subgraph (i.e. its components are exactly those of the subgraph).

**Lemma 3.13** (The two extremes).  $r_{\max} := \max_T \rho(T) = n - c(R)$  and  $r_{\min} := \min_T \rho(T) = c(B) - 1$ , and the trees  $T_{\max}, T_{\min}$  computed by the algorithm attain these values.

**Proof Sketch.** For any spanning tree  $T$ , the set  $T \cap R$  is a forest of  $(V, R)$ , so  $\rho(T) \leq n - c(R)$  by (ii). Conversely, a maximal forest  $F$  of  $(V, R)$  has exactly  $n - c(R)$  edges and extends to a spanning tree  $T$  with  $T \cap R \supseteq F$ , so  $\rho(T) \geq n - c(R)$ ; hence  $r_{\max} = n - c(R)$ . Kruskal run with all red edges offered before all blue ones adds red edges greedily until no red edge joins two different current components — i.e. it builds exactly a maximal forest of  $(V, R)$  — and then completes it with blue edges, so  $\rho(T_{\max}) = n - c(R) = r_{\max}$ . Symmetrically,  $|T \cap B| \leq n - c(B)$  for every spanning tree, i.e.  $\rho(T) = (n - 1) - |T \cap B| \geq (n - 1) - (n - c(B)) = c(B) - 1$ , and the blue-first Kruskal run makes  $T_{\min} \cap B$  a maximal forest of  $(V, B)$ , attaining  $\rho(T_{\min}) = c(B) - 1 = r_{\min}$ .

**Lemma 3.14** (One swap raises the red count by exactly one). *Let  $T$  be a spanning tree with  $\rho(T) < r_{\max}$ . Then there exists a red edge  $e \notin T$  whose fundamental cycle  $C_T(e)$  contains a blue tree edge  $f$ ; and for any such pair,  $T' = (T \setminus \{f\}) \cup \{e\}$  is a spanning tree with  $\rho(T') = \rho(T) + 1$ .*

**Proof Sketch.** The forest  $T \cap R$  of  $(V, R)$  has  $\rho(T) < n - c(R)$  edges, so by (ii) it is *not* a maximal forest of  $(V, R)$ : its components are not the components of  $(V, R)$ , and therefore some red edge  $e$  has its two endpoints in *different* components of  $T \cap R$ . Such an  $e$  cannot lie in  $T$  (otherwise  $e \in T \cap R$  and its endpoints would be in the same component of  $T \cap R$ ), so  $e \notin T$  and its fundamental cycle  $C_T(e)$  is well defined:  $C_T(e) \setminus \{e\}$  is the path of  $T$  joining the endpoints of  $e$ . If every edge of that path were red, the path would lie inside  $T \cap R$  and would connect the two endpoints there, contradicting the choice of  $e$ . Hence the path contains a blue edge  $f$ . By (i),  $T' = (T \setminus \{f\}) \cup \{e\}$  is a spanning tree; it discards one blue edge and adds one red edge, so  $\rho(T') = \rho(T) + 1$  exactly.

**Corollary 3.15** (Interval property).  $\{\rho(T) : T \text{ a spanning tree of } G\} = \{r_{\min}, r_{\min} + 1, \dots, r_{\max}\}$ .



**Proof Sketch.** No value outside  $[r_{\min}, r_{\max}]$  occurs, by definition of the minimum and the maximum. For the converse, start from  $T_{\min}$  and apply the previous lemma repeatedly: as long as the current tree has  $\rho < r_{\max}$  the lemma yields a spanning tree whose red count is larger by exactly one, so the trees produced realise every integer  $r_{\min}, r_{\min} + 1, \dots, r_{\max}$  in turn.

Consequently, if  $k \notin [r_{\min}, r_{\max}]$  then no spanning tree with exactly  $k$  red edges exists and the algorithm's early return is correct. If  $k \in [r_{\min}, r_{\max}]$ , the swap loop is exactly the repeated application of the one-swap lemma: at every iteration the current tree satisfies  $\rho(T) = r < k \leq r_{\max}$ , so the lemma guarantees that the scan over red non-tree edges does find one whose fundamental cycle contains a blue tree edge (the loop never gets stuck), and the swap performed keeps  $T$  a spanning tree while increasing  $r$  by exactly 1. After  $k - r_{\min}$  iterations the loop halts with a spanning tree having exactly  $k$  red edges. ■

### Problem 10: Lightest Path (Fewest Edges, Then Least Weight)

In a directed graph with positive edge weights, find the path from  $s$  to  $d$  that is lexicographically best under (number of edges, total weight), in  $O(m + n)$ .

**Algorithmic Intuition.** The primary criterion (fewest edges) means we should think in terms of BFS layers by edge-count distance from  $s$ . Within the layered structure, only edges that go from level  $\ell$  to level  $\ell + 1$  can possibly lie on any minimum-edge-count path; edges within a level or going backward are irrelevant to achieving the minimum length and can be discarded. This layered subgraph is a DAG (edges strictly increase level), so we can then run a simple one-pass DAG shortest-weight computation (secondary criterion) processing vertices level-by-level (a valid topological order of this layered DAG).

---

#### Algorithm 36: LightestPath( $G, s, d$ )

---

```

// Phase 1: BFS to compute levels (edge-count distance) from s
1 level[s] ← 0; level[v] ← ∞ for v ≠ s; Q ← [s];
2 while Q non-empty do
3   u ← Q.dequeue();
4   foreach edge (u, v) ∈ E do
5     if level[v] = ∞ then
6       level[v] ← level[u] + 1; enqueue v
// Phase 2: build layered DAG G' keeping only edges (u, v) with
//           level[v] = level[u] + 1
// Phase 3: process vertices in increasing order of level (a valid
//           topological order of G'), computing min weight to reach each at its
//           minimum level
7 weight[s] ← 0; weight[v] ← ∞ for v ≠ s;
8 foreach u in increasing order of level[u] do
9   foreach edge (u, v) ∈ G' do
10    if weight[u] + w(u, v) < weight[v] then
11      weight[v] ← weight[u] + w(u, v); pred[v] ← u;
12 return path reconstructed via pred from d back to s, and (level[d], weight[d]);

```

---

**Complexity Analysis.** Phase 1 (BFS):  $O(m + n)$ . Building  $G'$ : a single  $O(m)$  pass filtering edges by level. Phase 3: vertices can be bucketed by level during BFS (each vertex assigned to exactly one level bucket,  $O(n)$  total), and processing "in increasing level order" then costs  $O(n)$  for vertices plus  $O(m)$  for edges (each edge of  $G'$  examined once) — this is exactly a DAG relaxation pass,  $O(m + n)$ . Total:  $O(m + n)$ .

**Proof of Correctness.** *Claim 1: every minimum-edge-count path from  $s$  to any  $v$  uses only edges of  $G'$ .* By definition of BFS levels,  $\text{level}[v]$  equals the true minimum number of edges from  $s$  to  $v$  in  $G$  (standard BFS correctness). Any shortest (fewest-edge)  $s$ - $v$  path visits vertices at levels  $0, 1, 2, \dots, \text{level}[v]$  in strictly increasing order (since each edge of a shortest path must increase the level by exactly 1 — it cannot increase by more, else that vertex's true level would be smaller via this very path, contradicting level being the true minimum, and it cannot increase by 0 or decrease, else the path would not be of minimum length, as we could skip/shortcut). Hence every edge of a minimum-edge-count path satisfies  $\text{level}[v] = \text{level}[u] + 1$ , i.e. lies in  $G'$ .

*Claim 2:  $G'$  is a DAG and "increasing level order" is a valid topological order.* Every edge in  $G'$  strictly increases level by construction, so no cycle is possible (a cycle would require level to return to its starting value while strictly increasing at every step).

*Claim 3: Phase 3 correctly computes, for each  $v$ , the minimum total weight among all minimum-edge-count  $s$ - $v$  paths.* By Claim 1, restricting to  $G'$  loses no minimum-edge-count path and introduces no path shorter (in edge count) than  $\text{level}[v]$  (since  $G'$ -paths from  $s$  to  $v$  necessarily have exactly  $\text{level}[v]$  edges, one per level increment). Processing vertices in topological (level) order and relaxing exactly as in standard DAG shortest paths correctly computes, by a standard induction on level (identical to the classical DAG-shortest-path correctness proof: when  $u$  is processed, all edges into  $u$  have already been relaxed since their sources are at strictly smaller levels, so  $\text{weight}[u]$  is already final and correct before  $u$ 's outgoing edges are relaxed), the true minimum weight over all  $G'$ -paths, which by Claim 1 equals the true minimum weight over all true minimum-edge-count  $G$ -paths. Hence the returned pair  $(\text{level}[d], \text{weight}[d])$  is exactly the lexicographically optimal (length, weight) value, and the reconstructed path achieves it. ■

## Problem 11: Minimum Red-Blue Distance

Given a directed graph with positive weights and vertices colored red/blue/yellow, compute  $\min_{u \in R, v \in B} \delta(u, v)$  in  $O(m \log n)$ .

**Algorithmic Intuition.** We want the shortest distance from *any* red vertex to *any* blue vertex. Rather than running  $|R|$  separate single-source Dijkstra computations (too slow), add a virtual super-source  $s^*$  connected by zero-weight directed edges to every red vertex; a single Dijkstra run from  $s^*$  then computes  $\delta(s^*, v) = \min_{u \in R} \delta(u, v)$  for every  $v$  simultaneously, since the only way to reach  $v$  from  $s^*$  is via some red vertex  $u$  (the free entry point) followed by the true shortest  $u$ -to- $v$  path.

---

**Algorithm 37:** MinRedBlueDistance( $G = (V, E, w)$ ,  $R, B, Y$ )

---

- 1 Construct  $G^* = G$  with an added vertex  $s^*$  and edges  $(s^*, u)$  of weight 0 for every  $u \in R$ ;
  - 2 Run Dijkstra's algorithm from  $s^*$  on  $G^*$ , obtaining  $\delta(s^*, v)$  for all  $v \in V$ :  $O(m \log n)$ ;
  - 3 **return**  $\min_{v \in B} \delta(s^*, v)$ ;
- 

**Complexity Analysis.**  $G^*$  has  $m + |R| \leq m + n$  edges and  $n + 1$  vertices. Dijkstra with a binary heap runs in  $O((m + n) \log n) = O(m \log n)$  (assuming the graph is connected enough that

$m = \Omega(n)$ , standard for this bound; otherwise  $O((m+n)\log n)$  is the precise bound). The final scan over  $B$  is  $O(n)$ .

**Proof of Correctness.** Since all edge weights (original and the 0-weight super-source edges) are non-negative, Dijkstra's algorithm from  $s^*$  correctly computes true shortest-path distances  $\delta(s^*, v)$  for all  $v$ . Any path from  $s^*$  to  $v$  must begin with one of the zero-weight edges  $(s^*, u)$  for some  $u \in R$  (these are  $s^*$ 's only outgoing edges), followed by a path from  $u$  to  $v$  in the original graph  $G$ ; since the initial edge contributes 0 to the total weight,  $\delta(s^*, v) = \min_{u \in R} (0 + \delta_G(u, v)) = \min_{u \in R} \delta_G(u, v)$  exactly. Therefore

$$\min_{v \in B} \delta(s^*, v) = \min_{v \in B} \min_{u \in R} \delta_G(u, v) = \min_{u \in R, v \in B} \delta_G(u, v),$$

which is exactly the desired quantity. ■

## Problem 12: Desert Motorcycle Route with Fuel Constraint

Given a weighted undirected graph with fuel stations at a subset  $F \subseteq V$  (with  $s \in F$ ) and a tank range  $c$ , find the shortest feasible  $s$ - $d$  route (possibly revisiting vertices), where the vehicle must refuel at a station before exceeding range  $c$  since its last refuel, in  $O(mn \log n)$ .

**Algorithmic Intuition.** The key insight is that only the fuel-station vertices matter as "decision points": from any station, the vehicle can reach, on a single tank, any other station (or  $d$ ) whose *true shortest-path distance* (in the original graph, possibly passing through non-station junctions or even other stations en route) is  $\leq c$  — and it never hurts to consider that direct achievable distance as one "hop," since refueling only adds capability, never removes it. This reduces the problem to building a smaller auxiliary graph on just the stations (plus  $s, d$ ) and solving ordinary shortest paths on it.

---

**Algorithm 38:** FeasibleShortestRoute( $G = (V, E, w)$ ,  $F$  (stations,  $s \in F$ ),  $d$ ,  $c$ )

---

```

1  $F' \leftarrow F \cup \{d\};$ 
2 Initialize auxiliary graph  $H = (F', \emptyset);$ 
3 foreach station  $u \in F$  do
4   Run Dijkstra's algorithm from  $u$  on  $G$ :  $O(m \log n)$ , obtaining  $\delta_G(u, v)$  for all  $v \in V$ ;
5   foreach  $v \in F' \setminus \{u\}$  do
6     if  $\delta_G(u, v) \leq c$  then
7        $\quad$  add edge  $(u, v)$  to  $H$  with weight  $\delta_G(u, v);$ 
8 Run Dijkstra's algorithm from  $s$  on  $H$  to find  $\delta_H(s, d);$ 
9 if  $d$  is reachable from  $s$  in  $H$  then
10  return  $\delta_H(s, d)$  (the shortest feasible route length)
11 else
12  return "No feasible route exists"
```

---

**Complexity Analysis.**  $|F| \leq n$  Dijkstra runs on  $G$ , each  $O(m \log n)$ : total  $O(mn \log n)$ . Building  $H$ 's edges:  $O(n)$  per Dijkstra run,  $O(n^2)$  overall, dominated by the above. Final Dijkstra on  $H$  (which has  $\leq n$  vertices,  $\leq n^2$  edges):  $O(n^2 \log n)$ , also dominated by  $O(mn \log n)$  whenever  $m = \Omega(n)$  (true for any connected graph). Total:  $O(mn \log n)$ , matching the target.

## Proof of Correctness.

**Lemma 3.16.** *A route from  $s$  to  $d$  is feasible (respects the fuel constraint, refueling only at stations in  $F$ ) with total length  $L$  if and only if there is a walk  $s = u_0, u_1, \dots, u_k = d$  in  $H$  (as constructed above, with  $u_0, \dots, u_{k-1} \in F$ ,  $u_k = d$ ) with  $\sum_i w_H(u_{i-1}, u_i) = L$ .*

**Proof Sketch.** ( $\Leftarrow$ ) Given such a walk in  $H$ , each edge  $(u_{i-1}, u_i) \in H$  by construction has weight equal to the true  $G$ -shortest-path distance between  $u_{i-1}$  and  $u_i$ , which is  $\leq c$  (else the edge would not have been added). Concatenating the actual  $G$ -shortest paths realizing each hop yields an actual route in  $G$  from  $s$  to  $d$  of total length  $\sum_i w_H(u_{i-1}, u_i) = L$ ; this route is feasible because at the start of each hop  $u_{i-1} \in F$  is a station (full refuel available), and the hop's length is  $\leq c$ , so the tank suffices for that entire hop before the next refuel opportunity at  $u_i$  (also a station, or  $d$  at the final hop).

( $\Rightarrow$ ) Given any feasible route in  $G$  of length  $L$ , consider the maximal sub-segments between consecutive refuels (i.e., between consecutive visits to stations at which the vehicle actually refuels, including the mandatory start at  $s$  and implicitly ending at  $d$ ). Each such sub-segment, from station  $u_{i-1}$  to the next refuel point  $u_i \in F'$ , has length  $\leq c$  (else the route would not be feasible — it would run out of fuel), and its length is  $\geq \delta_G(u_{i-1}, u_i)$  (any actual path's length is at least the shortest-path distance between its endpoints). Replacing each such sub-segment by the true shortest path between its endpoints (which by the above is also  $\leq c$ , hence a valid edge of  $H$ , since shortest-path distance is  $\leq$  any particular path's length  $\leq c$ ) can only decrease or maintain the total length, yielding a walk in  $H$  of length  $\leq L$ . Hence the shortest  $H$ -walk length is  $\leq$  the shortest feasible  $G$ -route length; combined with the ( $\Leftarrow$ ) direction (every  $H$ -walk corresponds to an actual feasible  $G$ -route of equal length), the two shortest lengths are exactly equal.

Since Dijkstra's algorithm correctly computes  $\delta_H(s, d)$  (true shortest walk length in  $H$ , all weights  $\geq 0$ ), by the lemma this exactly equals the shortest feasible route length in  $G$ , and  $d$  unreachable in  $H$  correctly corresponds to no feasible route existing in  $G$  (by the contrapositive of the lemma's two directions). ■

**Worked Verification (from the problem statement).** Stations  $F = \{s, b, e\}$ ,  $c = 250$ ,  $d$  the destination. Running a bounded search from  $s$ :  $\delta_G(s, a) = 200$ ,  $\delta_G(s, b) = 200 + 40 = 240 \leq c$  (edge  $s$ - $b$  added, weight 240);  $\delta_G(s, d)$  via  $a$  is  $200 + 150 = 350 > c$  (no direct edge  $s$ - $d$ ). From  $b$ :  $\delta_G(b, e) = 200 \leq c$  (edge  $b$ - $e$ , weight 200);  $\delta_G(b, d)$  via  $a$  is  $40 + 150 = 190 \leq c$  (edge  $b$ - $d$ , weight 190). From  $e$ :  $\delta_G(e, d) = 200 \leq c$  (edge  $e$ - $d$ , weight 200). Running Dijkstra on  $H = \{s, b, e, d\}$  with these edges gives shortest  $s$ -to- $d$  path  $s \rightarrow b \rightarrow d$  of length  $240 + 190 = 430$ , matching the length of the example's stated optimal route  $s \rightarrow a \rightarrow b \rightarrow a \rightarrow d$  ( $200 + 40 + 40 + 150 = 430$ ) exactly, confirming the algorithm's correctness on this instance.